IACR Communications in Cryptology
ISSN 3006-5496, Vol. 2, No. 3, 38 pages.

https://doi.org/10.62056/angy11fgx

Check for updates

zkMaP: Zero-Knowledge Succinct Non-Interactive Matrix Multiplication Proofs

Biniyam Deressa and M. Anwar Hasan

University of Waterloo, Waterloo, Canada

Abstract.
We introduce *zkMaP* (Zero-Knowledge Succinct Non-Interactive Matrix Multiplication Proofs), a novel non-interactive zero-knowledge proof system for verifying matrix multiplication with significant improvements in efficiency and scalability. Our protocol leverages KZG polynomial commitments and an innovative inner-product reduction technique to reduce the verification of $n \times n$ matrix multiplication to a single pairing equation, thereby enabling constant-time verification independent of the matrix size. In particular, *zkMaP* requires only two pairing operations and produces proofs as small as 320 bytes, yielding a 96% reduction in proof size compared to prior schemes. Furthermore, the prover's computational complexity follows the state-of-the-art at $O(n^2)$, with experimental results demonstrating that proofs for 1024×1024 matrices can be generated in approximately 12.21 seconds, offering a $16.14\times$ speedup over previous methods. Our implementation also exhibits better memory efficiency, using only 24.58 MB of prover-side RAM for 1024×1024 matrices, and supports scalable batch processing, achieving per-proof generation times of 46.79 milliseconds for 1024 instances while maintaining a constant verification time of 3.6 ms.

Keywords: Zero-Knowledge Proofs · Succinct Non-Interactive Arguments · Matrix Multiplication · Polynomial Commitments · Batch Verification

1 Introduction

Matrix multiplication is a core computational primitive with wide-ranging applications in fields such as machine learning, data analytics, scientific computing, and cryptography. In an era where data-driven decision-making and collaborative computation underpin critical applications, ensuring both privacy and verifiability is more important than ever. Zero-knowledge proofs (ZKPs) have emerged as a transformative cryptographic tool, enabling a prover to demonstrate the validity of a statement to a verifier without disclosing any underlying secret information.

However, directly applying conventional ZKP systems (such as zk-SNARKs [BCCT12] or zk-STARKs [BSBHR18]) to matrix multiplication presents significant challenges. For large-scale datasets, succinctness becomes a critical concern: it is imperative that the proof size and verification time remain small, ideally constant or polylogarithmic in the size of the matrix. Furthermore, prover efficiency presents another substantial hurdle, as standard approaches that encode matrix multiplication in arithmetic circuits typically incur a prover complexity of $O(n^3)$ for $n \times n$ matrices, which is prohibitive for high-dimensional operations [XZZ+19, PHGR16]. Additionally, many applications require multiple, sequential matrix multiplications (e.g., in deep neural networks), necessitating

E-mail: bderessa@uwaterloo.ca (Biniyam Deressa), ahasan@uwaterloo.ca (M. Anwar Hasan)

Received: 2025-07-06 Accepted: 2025-09-02

commitment consistency where the same private inputs are consistently committed across different operations.

This work presents *zkMaP*, a specialized zero-knowledge protocol for matrix multiplication that achieves significant improvements over existing approaches. Our protocol leverages Kate-Zaverucha-Goldberg (KZG) polynomial commitments [KZG10] combined with an efficient inner-product reduction technique to transform matrix multiplication verification into a succinct polynomial identity check. *zkMaP* requires only two pairing operations and produces proofs as small as 320 bytes, yielding a 96% reduction in proof size compared to prior schemes. Furthermore, the prover's computational complexity follows the state-of-the-art at $O(n^2)$, with experimental results demonstrating that proofs for 1024×1024 matrices can be generated in approximately 12.21 seconds, offering a $16.14\times$ speedup over previous methods.

1.1 Paper Organization

The remainder of the paper is structured as follows. Section 1.2 surveys related work. Section 1.3 outlines our main contributions. Section 2 introduces the necessary algebraic background and notation. Section 3 defines the matrix multiplication proof problem and the corresponding security model. Section 4 presents the *zkMaP* protocol. Section 5 formally proves completeness, soundness, and zero-knowledge. Section 6 extends the protocol to support efficient batch verification. Section 7 analyzes computational costs. Section 8 reports experimental results. Section 9 concludes the paper.

1.2 Related Works

Zero-knowledge proofs (ZKPs) have evolved significantly over the past decade, driven by the need for privacy-preserving verification in a variety of applications. Early systems such as Pinocchio [PHGR16], Sonic, [MBKM19], Hyperplonk [CBBZ23], Groth16 [Gro16] demonstrated that succinct non-interactive zero-knowledge (zk-SNARK) proofs could achieve constant-size communication and polylogarithmic verification time. However, when these generic systems are applied to specific problems like matrix multiplication, their prover complexity scales poorly (typically as $O(n^3)$ for $n \times n$ matrices), which limits their practical utility.

To address these limitations, Thaler [Tha13] proposed a specialized protocol that reduces matrix multiplication into a sumcheck relation. By leveraging this transformation, the prover time can be reduced to $O(n^2)$. Unfortunately, this gain comes at the expense of verification efficiency, with verification time growing linearly with the matrix size. Building on this idea, LegoSNARK [CFQ19] further refines the approach by ensuring that the intermediate computations are consistent across multiple sequential operations. LegoSNARK achieves $O(n^2)$ prover time and polylogarithmic verifier time, which makes it more suitable for applications where consistency of secret inputs (such as in neural network layers) is critical. In contrast, Libra [XZZ+19] uses alternative cryptographic primitives to obtain improvements in transcript size and verification efficiency. However, Libra still suffers from relatively high prover overhead in certain cases, particularly when matrix dimensions increase.

QuickSilver [YSWW21] takes a distinct approach by employing interactive protocols and secure multiparty computation to achieve $O(n^2)$ complexity for both the prover and the verifier. Although this approach reduces the computational burden, it introduces non-negligible communication overhead and additional complexity in managing the interactive

rounds. Another line of work that has influenced the design of efficient ZK protocols is the use of inner-product arguments, most notably Bulletproofs [BBB+18]. Bulletproofs achieve logarithmic verifier time for range proofs and can be adapted to prove inner-product relations. However, their direct application to matrix multiplication is inefficient, as it requires multiple inner-product proofs and careful management of intermediate commitments to ensure consistency.

In parallel, universal and upgradable zk-SNARK constructions such as Sonic [MBKM19], Marlin [CHM+20], and PLONK [GWC19] have emerged. These protocols provide compelling features, including universal setup and efficient update mechanisms, but their generality often leads to reduced efficiency for specific operations such as matrix multiplication. The zkMatrix protocol [CYY24] distinguishes itself in this context by achieving $O(n^2)$ prover time for the multiplication of two $n \times n$ matrices while maintaining a succinct transcript and logarithmic verifier time. By employing batch processing, zkMatrix supports simultaneous proofs for multiple matrix multiplications with minimal additional overhead. The approach leverages Bulletproofs-inspired techniques to shift computational load from the verifier to the prover, making it highly efficient even in settings where multiple operations are performed sequentially.

Our zkMaP protocol further advances the state-of-the-art by using KZG polynomial commitments to achieve constant-size proofs and constant verification time, while maintaining the $O(n^2)$ prover time that matches the inherent complexity of matrix operations. A key distinction is that zkMaP operates directly on matrices without requiring arithmetic circuit representations, unlike Pinocchio [PHGR16], Groth16 [Gro16], LegoSNARK [CFQ19], and Libra [XZZ+19], which all necessitate transforming matrix multiplication into complex circuit constraints. This circuit-free approach eliminates the substantial overhead associated with circuit generation and evaluation. Furthermore, zkMaP is non-interactive and requires only logarithmic additional memory beyond the cost of storing matrices and the structured reference string. Our sub-batching technique for batch verification (Batch-zkMaP) allows processing multiple matrix multiplications with controlled memory usage of $O(n^2)$ regardless of the total batch size t, requiring only $O(tn^2)$ field multiplications plus $\lceil t/b \rceil$ exponentiations for t matrix pairs, where the configurable parameter b determines the memory-performance trade-off. As demonstrated in our implementation, zkMaP exhibits significant performance improvements over existing schemes in both proof generation and verification times, particularly for large matrices and batch verification operations.

Table 1 summarizes the computational complexity and proof characteristics of existing ZKP schemes for matrix multiplication, including our proposed approach using KZG commitments. (Note: The complexity bounds reflect the additional overhead for proof generation and verification; inherent costs such as storing the matrices or the full SRS are not included.)

1.3 Our Contributions

Our contributions are multi-fold. First, we present *zkMaP*, a novel zero-knowledge proof protocol for matrix multiplication that substantially advances the state of the art. Our *zkMaP* leverages a specialized polynomial representation with KZG commitments, achieving a 320-byte proof size at 128-bit security and a constant 3.6 ms verification time, independent of matrix dimensions. For 1024×1024 matrices, *zkMaP* delivers a 16.14× prover speedup over previous methods while reducing proof size by 96%. The protocol extends naturally to non-square matrices $\mathbf{A} \in \mathbb{F}_p^{m \times \ell}$ and $\mathbf{B} \in \mathbb{F}_p^{\ell \times n}$ while maintaining constant verification time and proof size, addressing a key limitation of existing approaches.

Secondly, for batch verification, we extend *zkMaP* with a random-linear-combination aggregation mechanism that scales efficiently without compromising its core advantages.

Our method employs structured challenge vectors and weighted polynomial aggregation to mitigate adversarial cancellation, thereby ensuring computational soundness even at large batch sizes. Batch *zkMaP* maintains constant verification time and proof size across all batch sizes. Processing a batch of 1,024 matrix multiplication instances, batch *zkMaP* achieves a per-proof generation time of 46.79 milliseconds.

Third, we present a comprehensive security analysis establishing a theoretical foundation for *zkMaP* under the q-Strong Diffie-Hellman (q-SDH) assumption. We prove that the protocol satisfies perfect completeness, special honest verifier zero-knowledge, and computational knowledge soundness. Any proof forgery would require solving the q-SDH problem with non-negligible probability. By linking polynomial degree bounds to protocol security, we ensure distinct matrices yield distinct polynomials while maintaining efficient verification. This framework guarantees *zkMaP*'s robustness against adversarial attacks without compromising performance.

Fourth, we provide comprehensive complexity analysis covering both computational and memory requirements for all protocol phases. Our analysis encompasses field operations, group exponentiations, pairing computations, and memory usage patterns, enabling practitioners to predict performance characteristics for their specific use cases. The analysis includes both theoretical bounds and empirical validation through extensive benchmarking across varying matrix dimensions and batch sizes.

Lastly, we make available our software implementation of the proposed protocols, developed in Rust using the Arkworks library with the BLS12-381 pairing-friendly curve, providing 128-bit security and exceptional memory efficiency. Our implementation incorporates several optimizations including parallel matrix-to-polynomial conversion, efficient sub-batching techniques, and optimized pairing operations. For 1024×1024 matrices, *zkMaP* requires only 24 MB of prover-side RAM, outperforming previous approaches. Moreover, our sub-batching technique decouples memory complexity from batch size, maintaining $O(bn^2)$ memory usage with a configurable sub-batch size parameter b. This approach yields memory footprints as low as 192 KB per proof, as demonstrated in experiments with 1,024 batches.

Table 1 provides a comprehensive comparison of *zkMaP* against existing zero-knowledge protocols for matrix multiplication, highlighting our protocol's advantages in proof size, verification efficiency, and non-interactive design.

Table 1: Comparison of Zero-Knowledge Proofs for Matrix Multiplication of $n \times n$ Matrices

Protocol	Proof Size	RAM Usage		Timing		Interactivity	Assumptions
		Prover	Verifier	Prover	Verifier		
Pinocchio [PHGR16]	$O(1)$	$O(n^3)$	$O(1)$	$O(n^3)$	$O(1)$	Non-Int.	q-PKE, q-SDH
Thaler [Tha13]	$O(\log n)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Hash Functions
LegoSNARK [CFQ19]	$O(\log n)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	Non-Int.	q-PKE, q-SDH
Libra [XZZ+19]	$O(\log^2 n)$	$O(n^3)$	$O(\log n)$	$O(n^3)$	$O(\log n)$	Non-Int.	DLP, Hash
QuickSilver [YSWW21]	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	Interactive	OLE, Hash
zkMatrix (single) [CYY24]	$O(\log n)$	$O(n^2)$	$O(\log n)$	$O(n^2)$	$O(\log n)$	Interactive	DLP
zkMatrix (batch, t) [CYY24]	$O(t \log n)$	$O(n^2 + tn)$	$O(t \log n)$	$O(n^2 + tn)^{\mathbb{E}} + O(tn^2)^{\mathbb{M}}$	$O(t \log n)$	Interactive	DLP
zkMaP (single)	**O(1) (320B)**	$\mathbf{O(n^2)}$	$\mathbf{O(1)}^{\ddagger}$	$\mathbf{O(n^2)}^{\mathbb{M}}$	$\mathbf{O(1)}^{\mathbb{E}}$	**Non-Int.**	**q-SDH, AGM**
zkMaP (batch, t)	**O(1) (320B)**	$\mathbf{O(n^2)}^{\dagger}$	$\mathbf{O(1)}^{\ddagger}$	$\mathbf{O(tn^2)}^{\mathbb{M}} + \lceil \frac{t}{b} \rceil\ \mathbf{O(1)}^{\mathbb{E}}$	$\mathbf{O(1)}^{\mathbb{E}}$	**Non-Int.**	**q-SDH, AGM**

† Sub-batching with fixed size b eliminates t-dependency in memory (e.g., b = 8)
‡ Verifier memory assumes one-time trusted setup parameters are preprocessed
$\mathbb{E}$ Exponentiation operations in pairing groups (cost $\approx \log_2 p$ multiplications)
$\mathbb{M}$ Field multiplications in $\mathbb{F}_p$
Assumptions: q-PKE (q-Power Knowledge of Exponent), q-SDH (q-Strong Diffie-Hellman), DLP (Discrete Logarithm Problem), OLE (Oblivious Linear Evaluation), Hash (Collision-resistant hash functions), AGM (Algebraic Group Model)

2 Preliminaries

2.1 Notation

Let $\mathbb{F}_p$ be a finite field of prime order $p \approx 2^\lambda$ for security parameter λ. $\omega \in \mathbb{F}_p$ is a primitive root of unity with $\text{ord}(\omega) \geq n$. Matrices are bold uppercase (e.g., $\mathbf{A} \in \mathbb{F}_p^{n\times n}$), vectors bold lowercase (e.g., $\mathbf{v}, \mathbf{y}_L, \mathbf{y}_R \in \mathbb{F}_p^n$), and scalars standard lowercase. Matrix indices start at 1: a_{ij} denotes row i, column j. Matrix multiplication is denoted by $\cdot$ (or $\times$), and inner products by $\langle \mathbf{a}, \mathbf{b} \rangle$ or $\mathbf{a}^\top \mathbf{b}$. Elliptic curve groups over $\mathbb{F}_p$ are denoted $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$, each cyclic of prime order p, with bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. Group generators are $G_1 \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$, with scalar multiplication written as sG. Group operations use additive notation. Polynomials are denoted $P(x), Q(x) \in \mathbb{F}_p[x]$ with degree $\deg(P)$, and matrix encodings as $P_{\mathbf{A}}(x) \in \mathbb{F}_p[x]$. Encoded values include $P_\mu(x), P_{\mathbf{a}_y}(x), P_{\mathbf{b}_y}(x) \in \mathbb{F}_p[x]$, with $W(x) \in \mathbb{F}_p[x]$ as the witness polynomial. Commitment functions are denoted $\mathsf{Com}(\cdot)$, and polynomial commitments as $V = P(s)G \in \mathbb{G}_1$. Random sampling from $\mathbb{F}_p$ is written as $\rho \xleftarrow{\$} \mathbb{F}_p$. For $n \times n$ matrices, $\ell = \max(\lceil n/4 \rceil, 2\lambda)$ basis vectors. Challenge scalars from Fiat-Shamir [FS86] transforms are denoted y, with related vectors $\mathbf{y}_L, \mathbf{y}_R$. Batch commitments are indicated with superscripts, e.g., V^{batch}. Algorithms are denoted with calligraphic letters: $\mathcal{A}$ (adversary), $\mathcal{P}$ (prover), $\mathcal{V}$ (verifier), $\mathcal{S}$ (simulator), and $\mathcal{E}$ (extractor). Adversarial advantage is $\text{Adv}_{\mathcal{A}}(\lambda)$, and negligible functions are $\text{negl}(\lambda)$. Computational indistinguishability is written as $\approx_c$. Interactions are expressed as $\langle \mathcal{P}(w), \mathcal{V} \rangle(x)$.

2.2 Cryptographic Building Blocks

To establish the foundation for the KZG polynomial commitment scheme, we formalize the necessary concepts in elliptic curve cryptography and bilinear pairings, emphasizing their role in constructing secure and efficient cryptographic protocols.

2.2.1 Elliptic Curves and Pairing-Friendly Groups

Let $\mathbb{F}_p$ denote a finite field of prime order p. An elliptic curve $E(\mathbb{F}_p)$ is defined by the Weierstrass equation $y^2 = x^3 + ax + b$, where $a, b \in \mathbb{F}_p$. The set of points on this curve forms an abelian group under a well-defined addition operation, with the point at infinity O serving as the identity element. Scalar multiplication, defined as repeated addition, plays a crucial role in elliptic curve cryptography, whose security relies on the hardness of the discrete logarithm problem (DLP) in these groups.

Definition 1 (Discrete Logarithm Assumption)**.** Let $\mathcal{G}$ be a group generation algorithm that outputs (G, p, g) where G is a cyclic group of prime order p with generator g. The discrete logarithm assumption states that for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$:

$$\Pr\left[(G, p, g) \leftarrow \mathcal{G}(1^\lambda); h \leftarrow G; a \leftarrow \mathcal{A}(G, p, g, h) : g^a = h\right] \leq \text{negl}(\lambda).$$

Pairing-based constructions require special elliptic curves that support efficiently computable bilinear maps. These pairing-friendly curves (e.g., BLS12-381 or BN curves) operate over subgroups $\mathbb{G}_1 \subseteq E(\mathbb{F}_p)$, $\mathbb{G}_2 \subseteq E(\mathbb{F}_{p^k})$, and a multiplicative target group $\mathbb{G}_T \subseteq \mathbb{F}_{p^k}^\times$, where k is the embedding degree.

Definition 2 (Bilinear Pairing)**.** A bilinear pairing is a map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ that satisfies bilinearity, non-degeneracy, and efficient computability. Bilinearity requires that for all $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, and scalars $a, b \in \mathbb{Z}_q$, it holds that $e(aP, bQ) = e(P, Q)^{ab}$. Non-degeneracy ensures that the pairing does not map all nontrivial elements to the identity, meaning that if $P \neq O$ and $Q \neq O$, then $e(P, Q) \neq 1_{\mathbb{G}_T}$. Efficient computability guarantees

that there exists a polynomial-time algorithm to evaluate $e(P, Q)$ for any valid inputs. These properties make bilinear pairings fundamental to numerous cryptographic protocols, including zero-knowledge proofs and polynomial commitment schemes.

In this work, we use an asymmetric Type-3 pairing setup, where $\mathbb{G}_1 \neq \mathbb{G}_2$ and no efficiently computable isomorphisms exist between these groups. This choice provides efficiency and stronger security properties for our commitment scheme.

2.2.2 Zero-Knowledge Arguments of Knowledge

Zero-knowledge arguments of knowledge (ZK-AoK) enable a prover to convince a verifier of a statement's validity without revealing the witness. Our matrix multiplication protocol uses these arguments to prove knowledge of matrices A and B while preserving their confidentiality.

Definition 3 (Zero-Knowledge Argument of Knowledge)**.** Let $R \subseteq \mathcal{X} \times \mathcal{W}$ be an NP relation, where $\mathcal{X}$ is the set of statements and $\mathcal{W}$ the set of witnesses. A protocol between a prover $\mathcal{P}$ and verifier $\mathcal{V}$ is a zero-knowledge argument of knowledge for relation R if:

1. *Completeness*: For all $(x, w) \in R$, $\Pr[\langle \mathcal{P}(x, w), \mathcal{V}(x) \rangle = \text{accept}] = 1$.
2. *Knowledge soundness*: For every PPT prover $\mathcal{P}^*$, there exists a PPT extractor $\mathcal{E}$ such that for any statement x, $\Pr[\langle \mathcal{P}^*(x), \mathcal{V}(x) \rangle = \text{accept} \wedge (x, w) \notin R \mid w \leftarrow \mathcal{E}(x)] \leq \text{negl}(\lambda)$.
3. *Zero-knowledge*: There exists a PPT simulator $\mathcal{S}$ such that for all $(x, w) \in R$, $\{\text{View}_{\mathcal{V}}[\langle \mathcal{P}(x, w), \mathcal{V}(x) \rangle]\} \approx_c \{\mathcal{S}(x)\}$, where $\text{View}_{\mathcal{V}}$ represents the verifier's view of the interaction and $\approx_c$ denotes computational indistinguishability.

A protocol is *succinct* if both the proof size and verification time are polynomial in the security parameter λ and logarithmic in the witness size $|w|$. Our KZG-based matrix multiplication protocol achieves succinctness with constant-size proofs and verification time independent of matrix dimensions.

2.3 The Kate-Zaverucha-Goldberg (KZG) Polynomial Commitment Scheme

Polynomial commitment schemes enable a prover to commit to a polynomial while allowing efficient verification of claimed evaluations at specific points. The KZG scheme [KZG10] provides a significant advancement in this domain by offering constant-sized commitments and succinct proofs through pairing-based cryptography. Its homomorphic properties support efficient proof aggregation, making it particularly valuable for verifiable computation, zero-knowledge proofs, and blockchain applications. In the commit-then-prove paradigm, KZG allows a prover to bind to a polynomial commitment and later prove statements about polynomial evaluations without revealing the entire polynomial.

2.3.1 Structured Reference String Setup

The KZG scheme requires a structured reference string (SRS) generated during a trusted setup phase. For a polynomial $P(x) = \sum_{i=0}^{d} c_i x^i \in \mathbb{F}_p[x]$ of degree d, the setup involves selecting a secret scalar $s \xleftarrow{\$} \mathbb{F}_p$ and publishing:

$$\mathsf{SRS} = \{G_1, sG_1, s^2G_1, \ldots, s^dG_1, G_2, sG_2\} \tag{1}$$

where $G_1 \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$ are group generators. To mitigate the risks of this "toxic waste" parameter, multi-party computation ceremonies are typically employed, ensuring

security as long as at least one participant remains honest. The SRS enables secure polynomial evaluation via group encodings: commitments $V = P(s)G_1$ allow verification of polynomial relations without revealing coefficients, while the bilinear structure supports efficient proof generation and verification. These encoded polynomials can represent various data structures including matrices and vectors, which we leverage in our protocol construction. The security of KZG commitments fundamentally relies on the computational hardness of the q-Strong Diffie–Hellman (q-SDH) assumption.

Definition 4 (q-Strong Diffie–Hellman Assumption)**.** Let $\mathcal{G}$ be a bilinear group generator that outputs $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, where p is prime, $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ are groups of order p, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a non-degenerate bilinear map. The q-Strong Diffie–Hellman (q-SDH) assumption states that for any PPT adversary $\mathcal{A}$, the following probability is negligible in the security parameter λ:

$$\Pr\left[\begin{array}{l}(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \mathcal{G}(1^\lambda);\\ G_1 \xleftarrow{\$} {\mathbb{G}_1}^*;\\ s \xleftarrow{\$} \mathbb{F}_p;\\ (c, (s+c)^{-1}G_1) \leftarrow \mathcal{A}(G_1, sG_1, s^2G_1, \ldots, s^qG_1)\end{array}\right] \leq \text{negl}(\lambda)$$

In other words, the q-SDH assumption ensures the binding property of KZG commitments: no adversary, even after observing up to q powers of the secret s, can open a commitment to two distinct polynomials.

2.3.2 Commitment and Verification

To commit to polynomial $P(x) \in \mathbb{F}_p[x]$, the prover computes $V_P = P(s)G_1 = \sum_{i=0}^{d} c_i(s^iG_1)$, yielding a single group element in $\mathbb{G}_1$ regardless of degree d. For a claimed evaluation $P(z) = y$, the prover constructs quotient polynomial $Q(x) = \frac{P(x)-y}{x-z} \in \mathbb{F}_p[x]$ and computes proof $\pi = Q(s)G_1$. Verification involves checking:

$$e(\pi, sG_2 - zG_2) \stackrel{?}{=} e(V_P - yG_1, G_2) \tag{2}$$

This equation holds for honest provers due to bilinearity: $e(\pi, sG_2 - zG_2) = e(G_1, G_2)^{Q(s)(s-z)} = e(G_1, G_2)^{P(s)-y} = e(V_P - yG_1, G_2)$. The scheme provides several key properties: (i) succinctness—constant-sized commitments and proofs; (ii) efficiency—verification requires only two pairings; (iii) optional hiding—information-theoretic hiding when combined with randomization; and (iv) binding—distinct polynomials yield distinct commitments under the q-SDH assumption. These properties make KZG foundational for many succinct proof systems.

2.4 Proof Aggregation via Homomorphism

KZG commitments possess a valuable homomorphic property that enables efficient aggregation of multiple proofs. For polynomials $P(x), Q(x)$ and scalars α, β, we have:

$$\mathsf{Com}(\alpha P + \beta Q) = \alpha \cdot \mathsf{Com}(P) + \beta \cdot \mathsf{Com}(Q) \tag{3}$$

This property forms the foundation for batch verification, allowing us to combine multiple matrix multiplication proofs into a single verification equation. In our construction, each proof instance includes commitments $V_{A,i}, V_{B,i}, V_{C,i}$ to matrix polynomials, alongside a witness π_i. To aggregate t such proofs, we derive a random challenge $\rho \xleftarrow{\$} \mathbb{F}_p$ via the

Fiat-Shamir transform and compute:

$$\begin{aligned} V_{\mathbf{A}}^{\text{agg}} &= \sum_{i=1}^{t} \rho^{i-1} \cdot V_{A,i}, \quad V_{\mathbf{B}}^{\text{agg}} = \sum_{i=1}^{t} \rho^{i-1} \cdot V_{B,i}, \\ V_{\mathbf{C}}^{\text{agg}} &= \sum_{i=1}^{t} \rho^{i-1} \cdot V_{C,i}, \quad \pi^{\text{agg}} = \sum_{i=1}^{t} \rho^{i-1} \cdot \pi_i \end{aligned} \tag{4}$$

Similarly, we aggregate evaluation points and values: $z^{\text{agg}} = \sum_{i=1}^{t} \rho^{i-1} \cdot z_i$ and $y^{\text{agg}} = \sum_{i=1}^{t} \rho^{i-1} \cdot y_i$. This random linear combination preserves soundness via the Schwartz-Zippel lemma. The aggregated verification check becomes:

$$e(\pi^{\text{agg}}, sG_2 - z^{\text{agg}} G_2) \stackrel{?}{=} e(V_{\mathbf{C}}^{\text{agg}} - y^{\text{agg}} G_1, G_2) \tag{5}$$

The correctness of this approach follows from the linearity of commitment and witness structures. Soundness is guaranteed by the unpredictability of ρ, if any individual proof is invalid, the aggregated proof fails verification with overwhelming probability. This batching technique transforms verification complexity from $O(t)$ pairing operations to just $O(1)$, enabling efficient verification of multiple matrix multiplication relations simultaneously.

2.5 Matrix Polynomial Encodings

Following Wahby et al. [WTS+18], we utilize structured polynomial encodings to transform matrix multiplication verification into polynomial identity checking. For a matrix $\mathbf{A} \in \mathbb{F}_p^{n \times n}$, we define its polynomial encoding as:

$$P_{\mathbf{A}}(x) = \sum_{i=1}^{n} \sum_{j=1}^{n} a_{ij}\, x^{(i-1)\cdot n + (j-1)} \in \mathbb{F}_p[x] \tag{6}$$

This encoding establishes a bijection between $n \times n$ matrices and polynomials of degree at most $n^2 - 1$, where each matrix entry corresponds to a unique monomial coefficient. Evaluation of the polynomial at a point $s \in \mathbb{F}_p$ requires $O(n^2)$ operations. By committing to $P_{\mathbf{A}}(x)$ using the KZG scheme, we obtain $V_{\mathbf{A}} = P_{\mathbf{A}}(s)G_1$, a single group element representation of the entire matrix.

The soundness of this approach is founded on the Schwartz-Zippel lemma [Sch80], which ensures that for distinct polynomials $P_{\mathbf{A}}(x)$ and $P_{\mathbf{B}}(x)$, the probability they evaluate to the same value at a random point $\alpha \in \mathbb{F}_p$ is bounded by:

$$\Pr\left[P_{\mathbf{A}}(\alpha) = P_{\mathbf{B}}(\alpha)\right] \leq \frac{n^2 - 1}{p} \tag{7}$$

With $p \geq 2^{\lambda}$, this probability becomes negligible in the security parameter λ. This encoding mechanism forms the algebraic foundation of our protocol, enabling efficient reduction of matrix relations to polynomial equations while ensuring succinct and verifiable proofs.

2.6 Matrix Multiplication Verification via Inner-Product Reduction

Verifying matrix multiplication $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ directly requires checking n^2 entries, leading to $O(n^3)$ complexity. Building on the work of Groth [Gro09] and zkMatrix [CYY24], we

reduce this verification to checking structured polynomial identities through inner-product decompositions. The key insight is to transform matrix multiplication into an efficient identity check using carefully constructed challenge vectors:

$$\mathbf{y}_L = \left(1, y^n, y^{2n}, \ldots, y^{(n-1)n}\right)^\top \in \mathbb{F}_p^n, \quad \mathbf{y}_R = \left(1, y, y^2, \ldots, y^{n-1}\right)^\top \in \mathbb{F}_p^n, \tag{8}$$

where $y \in \mathbb{F}_p$ is sampled uniformly at random. The specific construction of $\mathbf{y}_L$ and $\mathbf{y}_R$ ensures that each matrix entry contributes a unique monomial degree in the resulting polynomial, enabling detection of any discrepancy in the matrix multiplication. Using these vectors, the verification process becomes:

$$\mathbf{y}_L^\top \mathbf{C} \mathbf{y}_R \stackrel{?}{=} \left(\mathbf{y}_L^\top \mathbf{A}\right) \cdot \left(\mathbf{B}\mathbf{y}_R\right). \tag{9}$$

When $\mathbf{C} = \mathbf{A} \times \mathbf{B}$, both sides yield identical polynomials in y. Otherwise, they represent distinct polynomials of degree at most $n^2 - 1$. By the Schwartz-Zippel lemma, the probability of false acceptance is bounded by $(n^2 - 1)/p$, which is negligible for sufficiently large p.

This approach achieves succinctness by reducing matrix verification to a single polynomial identity check, provides soundness through randomized challenges, and integrates seamlessly with polynomial commitment schemes. While zkMatrix employs Bulletproof-based vector commitments, our construction utilizes KZG polynomial commitments, offering more efficient encoding and verification via pairing-based checks. This algebraic structure forms the foundation of our *zkMaP* protocol, detailed in Section 4.

3 Problem Formalization

This section provides a comprehensive formalization of the zero-knowledge matrix multiplication problem, establishing the formal problem statement, security requirements, and cryptographic assumptions underlying our protocol. Building upon the foundations from Section 2, we specify the framework that our *zkMaP* protocol addresses.

3.1 Formal Problem Definition

Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_p^{n \times n}$ be matrices over a finite field $\mathbb{F}_p$, where p is a prime of bitlength λ. For generality, our formulation extends to rectangular matrices $\mathbf{A} \in \mathbb{F}_p^{m \times \ell}$, $\mathbf{B} \in \mathbb{F}_p^{\ell \times n}$, and $\mathbf{C} \in \mathbb{F}_p^{m \times n}$ where the inner dimensions match for matrix multiplication compatibility. The goal is to design a zero-knowledge proof protocol that enables a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ of the correctness of the matrix multiplication relation $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ without revealing any information about the secret matrices $\mathbf{A}$ and $\mathbf{B}$ beyond the validity of this relation.

We formalize the matrix multiplication relation through the following NP-language:

$$\mathcal{R}_{\text{MatMul}} = \left\{ (V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}) \in \mathbb{G}_1^3 \;\middle|\; \begin{array}{l} \exists\, \mathbf{A}, \mathbf{B} \in \mathbb{F}_p^{n \times n} \text{ s.t. } \mathbf{C} = \mathbf{A} \times \mathbf{B} \text{ and} \\ V_{\mathbf{A}} = \mathsf{Com}(\mathbf{A}), V_{\mathbf{B}} = \mathsf{Com}(\mathbf{B}), V_{\mathbf{C}} = \mathsf{Com}(\mathbf{C}) \end{array} \right\}$$

where $\mathsf{Com} : \mathbb{F}_p^{n \times n} \to \mathbb{G}_1$ denotes the commitment function instantiated via the KZG polynomial commitment scheme described in Section 2.3. A zero-knowledge proof system for $\mathcal{R}_{\text{MatMul}}$ is defined by PPT algorithms:

$$\begin{aligned} \mathsf{Setup}(1^\lambda, n) &\to \text{pp} \\ \mathsf{Prove}(\text{pp}, \mathbf{A}, \mathbf{B}, \mathbf{C}) &\to \pi \\ \mathsf{Verify}(\text{pp}, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) &\to \{0, 1\} \end{aligned}$$

with formal definitions provided in Section 4.

The proof system demonstrates membership in $\mathcal{R}_{\text{MatMul}}$ by reducing the verification of matrix multiplication to efficiently checkable polynomial and inner-product relations, leveraging the algebraic structure of matrix operations while maintaining cryptographic security guarantees.

3.2 Security Requirements and Threat Model

A cryptographically sound zero-knowledge proof system for matrix multiplication must satisfy three fundamental security properties, as established in the foundational work of Goldreich and Oren [GO94]. Our security analysis addresses both the theoretical guarantees and practical deployment requirements essential for large-scale matrix multiplication verification scenarios.

Completeness: An honest prover with valid matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ satisfying $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ must always convince an honest verifier. Formally, for all security parameters $\lambda \in \mathbb{N}$, matrix dimensions $n \in \mathbb{N}$, and matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_p^{n \times n}$ with $\mathbf{C} = \mathbf{A} \times \mathbf{B}$:

$$\Pr\left[\mathsf{Verify}(pp, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) = 1 \;\middle|\; \begin{array}{l} pp \leftarrow \mathsf{Setup}(1^\lambda, n), \\ V_{\mathbf{A}} = \mathsf{Com}(\mathbf{A}), V_{\mathbf{B}} = \mathsf{Com}(\mathbf{B}), V_{\mathbf{C}} = \mathsf{Com}(\mathbf{C}), \\ \pi \leftarrow \mathsf{Prove}(pp, \mathbf{A}, \mathbf{B}, \mathbf{C}) \end{array}\right] = 1$$

This property requires that our polynomial encoding preserves the algebraic structure of matrix multiplication and that the commitment scheme maintains proper binding properties under the specified cryptographic assumptions.

Knowledge Soundness: Any computationally bounded adversary capable of producing a convincing proof with non-negligible probability must possess extractable knowledge of matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ that satisfy the multiplication relation. Formally, for every probabilistic polynomial-time algebraic adversary $\mathcal{A}$, there exists a probabilistic polynomial-time extractor $\mathcal{E}$ such that:

$$\Pr\left[\begin{array}{c} \mathsf{Verify}(pp, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) = 1 \\ \wedge\, (\mathbf{A}', \mathbf{B}', \mathbf{C}') \notin \mathcal{R}_{\text{MatMul}} \end{array} \;\middle|\; \begin{array}{l} pp \leftarrow \mathsf{Setup}(1^\lambda, n), \\ (V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) \leftarrow \mathcal{A}(pp), \\ (\mathbf{A}', \mathbf{B}', \mathbf{C}') \leftarrow \mathcal{E}^{\mathcal{A}}(pp, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) \end{array}\right] \leq \mathsf{negl}(\lambda)$$

where $\mathcal{E}^{\mathcal{A}}$ denotes the extractor with oracle access to $\mathcal{A}$'s algebraic representation. The extractor leverages the adversary's computational structure within the Algebraic Group Model [FKL18], exploiting the fact that all group elements output by $\mathcal{A}$ must be expressible as linear combinations of the structured reference string elements.

Zero-Knowledge: The verification process reveals no information about the private matrices $\mathbf{A}$ and $\mathbf{B}$ beyond the validity of $\mathbf{C} = \mathbf{A} \times \mathbf{B}$. This property is formalized through the existence of a probabilistic polynomial-time simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that for all probabilistic polynomial-time distinguishers $\mathcal{D}$ and matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C}) \in \mathcal{R}_{\text{MatMul}}$:

$$|\Pr[\mathcal{D}(pp, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) = 1] - \Pr[\mathcal{D}(pp, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi') = 1]| \leq \mathsf{negl}(\lambda)$$

where the real distribution involves honest proof generation $\pi \leftarrow \mathsf{Prove}(pp, \mathbf{A}, \mathbf{B}, \mathbf{C})$ and the ideal distribution uses simulated proofs $\pi' \leftarrow \mathcal{S}_2(\tau, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}})$ with simulation trapdoor τ. The protocol achieves *special honest-verifier zero-knowledge* (SHVZK) in the interactive setting, and *computational zero-knowledge* in the non-interactive setting via the Fiat–Shamir transformation [FS86] in the Random Oracle Model.

Hiding Properties: The privacy of matrix entries is ensured through the computational hiding property of KZG polynomial commitments, which relies on the discrete logarithm

assumption in the underlying bilinear groups. For any polynomial $P(x) = \sum_{i=0}^{d} p_i x^i$, the commitment $V_P = P(s)G_1$ computationally hides the coefficients $\{p_i\}$ under the discrete logarithm assumption, since extracting these coefficients from V_P would require solving the discrete logarithm problem. Matrix entries remain hidden through commitment randomness in the polynomial encoding, where each matrix element contributes to multiple polynomial coefficients, ensuring that no individual entry can be recovered from the commitment values. The zero-knowledge simulator generates statistically indistinguishable proof transcripts without access to the secret matrices by sampling random polynomial coefficients and computing appropriate commitment values, formally establishing that no information about $\mathbf{A}$ and $\mathbf{B}$ beyond the validity of the multiplication relation is revealed during verification.

Beyond cryptographic security, practical deployment requires additional efficiency properties that distinguish our approach from general-purpose zero-knowledge systems. The protocol achieves succinctness with constant proof size $O(1)$ independent of matrix dimension n, enabling efficient verification through exactly two pairing operations regardless of input size. The prover computation scales as $O(n^2)$, matching the fundamental cost of matrix multiplication while providing cryptographic verification guarantees.

Our threat model encompasses malicious provers who may attempt to violate soundness by convincing verifiers of incorrect matrix multiplication relations, providing inconsistent commitments that do not correspond to valid matrix multiplications, or deviating arbitrarily from the protocol specification. We also account for honest-but-curious verifiers who follow the protocol correctly but may attempt to extract additional information about private matrices beyond the validity of the relation. The zero-knowledge property ensures that such attempts yield no additional information.

The security analysis relies on well-established cryptographic assumptions, with the primary foundation being the q-Strong Diffie-Hellman (q-SDH) assumption [BLS04] with $q = n^2 - 1$. This assumption ensures the binding property of polynomial commitments [KZG10] and enables the knowledge extraction techniques essential for establishing soundness. The knowledge soundness analysis operates in the Algebraic Group Model (AGM), where adversaries must provide algebraic representations for all group elements they output, enabling efficient extraction by leveraging the adversary's computational structure. For the non-interactive version obtained via Fiat-Shamir transformation, security additionally relies on modeling the hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_p$ as a random oracle, ensuring collision resistance and unpredictability of verification challenges.

Additional structural requirements include field size constraints where the prime p satisfies $p \geq 2^\lambda$ to ensure negligible error probabilities in Schwartz-Zippel-based polynomial identity arguments, structured reference string support for polynomials of degree up to $n^2 - 1$ to accommodate the matrix encoding scheme, and underlying bilinear group instantiations that provide λ bits of security against best-known discrete logarithm attacks. The trusted setup phase assumes honest generation of the structured reference string with secure deletion of the secret trapdoor, achievable in practice through multi-party computation ceremonies [BGM17] where security is maintained as long as at least one participant acts honestly. This security framework strikes a balance between theoretical rigor and practical efficiency, offering robust cryptographic guarantees while preserving the performance characteristics necessary for large-scale matrix multiplication verification scenarios. The formal security analysis with complete definitions, theorems, and proofs is provided in Section 5.

4 zkMaP Protocol Specification

In this section, we present our Zero-Knowledge Matrix Product (zkMaP) protocol, which enables a prover to demonstrate that $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ for matrices defined over a finite field $\mathbb{F}_p$, while keeping both input matrices $\mathbf{A}$ and $\mathbf{B}$ private. Building upon the KZG polynomial commitment scheme described in Section 2.3, we detail the polynomial encoding of matrices, proof generation, and verification procedures.

4.1 Polynomial Encoding and Trusted Setup

We encode matrices as univariate polynomials to leverage the properties of KZG commitments. While our construction naturally extends to non-square matrices $\mathbf{A} \in \mathbb{F}_p^{m \times \ell}$ and $\mathbf{B} \in \mathbb{F}_p^{\ell \times n}$ yielding $\mathbf{C} \in \mathbb{F}_p^{m \times n}$, we focus on square matrices for clarity. For a matrix $M \in \mathbb{F}_p^{n \times n}$, we define its polynomial encoding as:

$$P_M(x) = \sum_{i=1}^{n} \sum_{j=1}^{n} m_{ij} x^{(i-1)n+(j-1)}.$$

This encoding creates a bijection between $n \times n$ matrices and polynomials of degree at most $n^2 - 1$. Crucially, it also preserves matrix multiplication under appropriate degree truncation.

Theorem 1 (Matrix Embedding Correctness and Soundness)**.** *Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_p^{n \times n}$ with polynomial encodings $P_{\mathbf{A}}(x)$, $P_{\mathbf{B}}(x)$, and $P_{\mathbf{C}}(x)$ defined by*

$$P_{\mathbf{M}}(x) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} m_{i+1,j+1}\, x^{in+j}.$$

Then:

1. ***Bijectivity**: The mapping $\phi : \mathbf{M} \mapsto P_{\mathbf{M}}(x)$ is a bijection between $\mathbb{F}_p^{n \times n}$ and the set of polynomials of degree at most $n^2 - 1$ over $\mathbb{F}_p$.*
2. ***Multiplication Preservation**: If $\mathbf{C} = \mathbf{A} \times \mathbf{B}$, then $P_{\mathbf{C}}(x) = P_{\mathbf{A} \times \mathbf{B}}(x)$.*
3. ***Soundness**: For $\mathbf{C} \neq \mathbf{A} \times \mathbf{B}$ and $s \xleftarrow{\$} \mathbb{F}_p$, $\Pr\left[P_{\mathbf{C}}(s) = P_{\mathbf{A} \times \mathbf{B}}(s)\right] \leq \frac{n^2-1}{p}$.*

Proof. We prove the correctness and soundness of the matrix embedding by addressing each claim in turn. The proof is organized as follows:

1. *Bijectivity***:** First, we show that the encoding function $\phi : \mathbb{F}_p^{n \times n} \to \mathbb{F}_p[x]$ is a bijection. Specifically, we prove that ϕ is both injective and surjective.

Injectivity: Suppose that $\phi(\mathbf{M}_1) = \phi(\mathbf{M}_2)$ for two matrices $\mathbf{M}_1, \mathbf{M}_2 \in \mathbb{F}_p^{n \times n}$. This implies that their polynomial encodings are identical, i.e., $P_{\mathbf{M}_1}(x) = P_{\mathbf{M}_2}(x)$. Since the encoding maps each matrix entry $m_{i+1,j+1}$ to a unique monomial x^{in+j}, matching the coefficients of the corresponding powers of x leads to the conclusion that $\mathbf{M}_1 = \mathbf{M}_2$.

Surjectivity: Given any polynomial $Q(x) = \sum_{k=0}^{n^2-1} q_k x^k$ over $\mathbb{F}_p$, we can define a matrix $\mathbf{Q} \in \mathbb{F}_p^{n \times n}$ where the entry at position (i, j) is q_{in+j}. This shows that every polynomial of degree at most $n^2 - 1$ is the encoding of some matrix, thus proving that ϕ is surjective.

2. *Multiplication Preservation***:** Now, we prove that the matrix multiplication operation is preserved under the encoding. Suppose that $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ for matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_p^{n \times n}$.

The polynomial encoding of $\mathbf{C}$ is given by:

$$P_{\mathbf{C}}(x) = \sum_{i,j=0}^{n-1} \left(\sum_{k=1}^{n} a_{i+1,k} b_{k,j+1} \right) x^{in+j}.$$

By rearranging the sums, we observe that this is equivalent to:

$$P_{\mathbf{C}}(x) = \sum_{k=1}^{n} \left(\sum_{i=0}^{n-1} a_{i+1,k} x^{in} \right) \left(\sum_{j=0}^{n-1} b_{k,j+1} x^{j} \right),$$

which is exactly $P_{\mathbf{A}\times\mathbf{B}}(x)$. Thus, we have shown that the polynomial encoding of $\mathbf{C}$ matches the encoding of the matrix product $\mathbf{A} \times \mathbf{B}$.

3. Soundness**:** Finally, we establish the soundness of the matrix embedding. Suppose that $\mathbf{C} \neq \mathbf{A} \times \mathbf{B}$. Then, the difference between the polynomials $P_{\mathbf{C}}(x)$ and $P_{\mathbf{A}\times\mathbf{B}}(x)$ is a non-zero polynomial of degree at most $n^2 - 1$. By the Schwartz-Zippel lemma, the probability that these polynomials are equal at a random point $s \in \mathbb{F}_p$ is bounded by:

$$\Pr_{s \xleftarrow{\$} \mathbb{F}_p} [P_{\mathbf{C}}(s) = P_{\mathbf{A}\times\mathbf{B}}(s)] \leq \frac{n^2 - 1}{p}.$$

Thus, the soundness of the matrix embedding follows, and this completes the proof of Theorem 1. □

Our encoding naturally generalizes to non-square matrices. For $\mathbf{A} \in \mathbb{F}_p^{m\times\ell}$ and $\mathbf{B} \in \mathbb{F}_p^{\ell\times n}$, we define:

$$P_{\mathbf{A}}(x) = \sum_{i=0}^{m-1} \sum_{j=0}^{\ell-1} a_{i+1,j+1} \cdot x^{i\ell+j}, \quad P_{\mathbf{B}}(x) = \sum_{i=0}^{\ell-1} \sum_{j=0}^{n-1} b_{i+1,j+1} \cdot x^{in+j}$$

The multiplication preservation property extends naturally, with prover cost $O(m\ell + \ell n)$ and verification remaining constant.

For example, consider matrices $\mathbf{A} \in \mathbb{F}_p^{2\times3}$ and $\mathbf{B} \in \mathbb{F}_p^{3\times2}$ with entries:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{pmatrix}$$

The polynomial encodings become:

$$P_{\mathbf{A}}(x) = a_{11} + a_{12}x + a_{13}x^2 + a_{21}x^3 + a_{22}x^4 + a_{23}x^5$$
$$P_{\mathbf{B}}(x) = b_{11} + b_{12}x + b_{21}x^2 + b_{22}x^3 + b_{31}x^4 + b_{32}x^5$$

For $\mathbf{C} = \mathbf{AB} \in \mathbb{F}_p^{2\times2}$, the encoding satisfies $P_{\mathbf{C}}(x) \equiv P_{\mathbf{A}}(x) \cdot P_{\mathbf{B}}(x) \pmod{x^4}$.

To support these polynomial operations, the *zkMaP* protocol requires a KZG structured reference string (SRS) with sufficient powers to handle matrices of dimension $n \times n$. For non-square matrices, the SRS must support polynomials of degree up to $\max(m\ell, \ell n, mn) - 1$. Specifically, the SRS must support polynomials of degree up to $n^2 - 1$:

$$\text{SRS} = \{G_1, sG_1, s^2G_1, \ldots, s^{n^2-1}G_1, G_2, sG_2\}, \tag{10}$$

where s is the secret value generated during the trusted setup process described in Section 2.3. The KZG commitment scheme ensures that commitments to polynomials of this form are binding under standard cryptographic assumptions.

Using these encodings and the KZG commitment scheme, the prover commits to matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ as:

$$V_{\mathbf{A}} = P_{\mathbf{A}}(s)G_1, \quad V_{\mathbf{B}} = P_{\mathbf{B}}(s)G_1, \quad V_{\mathbf{C}} = P_{\mathbf{C}}(s)G_1. \tag{11}$$

These polynomial-based commitments, combined with the inner-product reduction technique described in Section 2.6, form the basis of our succinct zero-knowledge protocol.

4.2 Non-Interactive Proof Generation

To transform the interactive protocol into a non-interactive zero-knowledge proof, we apply the Fiat–Shamir heuristic [FS86]. The prover first computes commitments $V_{\mathbf{A}}$, $V_{\mathbf{B}}$, and $V_{\mathbf{C}}$ for matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ as defined in Section 4. A challenge value $y \in \mathbb{F}_p$ is derived by applying a cryptographic hash function to all public parameters and commitments:

$$y = \mathcal{H}(\text{pp} \parallel V_{\mathbf{A}} \parallel V_{\mathbf{B}} \parallel V_{\mathbf{C}}) \bmod p, \tag{12}$$

where pp denotes public parameters including matrix dimensions and SRS description.

The prover then constructs structured vectors based on the challenge:

$$\mathbf{y}_L = (1, y^n, y^{2n}, \ldots, y^{(n-1)n})^\top, \quad \mathbf{y}_R = (1, y, y^2, \ldots, y^{n-1})^\top. \tag{13}$$

These vectors ensure each matrix entry is multiplied by a unique power of y, preventing coefficient collisions during aggregation.

The prover computes the projections:

$$\mathbf{a}_y = \mathbf{y}_L^\top \mathbf{A}, \quad \mathbf{b}_y = \mathbf{B}\mathbf{y}_R, \quad \mu = \mathbf{y}_L^\top \mathbf{C}\mathbf{y}_R$$

Note that $\mu = \mathbf{a}_y \cdot \mathbf{b}_y$ holds iff $\mathbf{C} = \mathbf{A} \times \mathbf{B}$. The prover encodes these as polynomials and computes commitments:

$$V_{\mathbf{a}_y} = P_{\mathbf{a}_y}(s)G_1, \quad V_{\mathbf{b}_y} = P_{\mathbf{b}_y}(s)G_1, \quad V_\mu = P_\mu(s)G_1$$

Finally, the prover computes the witness polynomial and its commitment:

$$W(x) = \frac{P_\mu(x) - P_{\mathbf{a}_y}(x) \cdot P_{\mathbf{b}_y}(x)}{x - y}, \quad V_W = W(s)G_1. \tag{14}$$

The proof π consists of the commitments $(V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, V_{\mathbf{a}_y}, V_{\mathbf{b}_y}, V_\mu, V_W)$.

4.3 Matrix Compression

While our polynomial embedding elegantly captures matrix structure, direct application faces scalability challenges: committing to an $n \times n$ matrix requires degree-$(n^2 - 1)$ polynomials and correspondingly large structured reference strings with $O(n^2)$ group elements. For practical matrices of dimension $n = 1024$, this translates to over one million SRS elements, making the approach prohibitively expensive. Our compression strategy employs Vandermonde matrix structure to create a compact, collision-resistant projection that reduces the effective polynomial degree from $O(n^2)$ to $O(\ell n)$ where $\ell = \max\big(\lceil n/4 \rceil, 2\lambda\big)$, preserving soundness with collision probability $< 2^{-\lambda}$ while enabling practical efficiency.

For security parameter λ and prime $p > 2^{2\lambda}n$, we uniformly sample $\omega \in \mathbb{F}_p^\times$ such that ω has multiplicative order at least n, generating a subgroup of size $\geq n$. The compressed representation $\mathbf{M}_{(\text{comp})} = \Phi(\mathbf{M}) \in \mathbb{F}_p^{\ell \times n}$ (where $\mathbf{M}$ denotes any input matrix) is constructed by evaluating column polynomials at carefully chosen evaluation points. For each column j, we form the polynomial $f_j(X) = \sum_{k=0}^{n-1} \mathbf{M}[k,j]X^k$ and evaluate it at powers of ω:

$$\mathbf{M}_{(\text{comp})}[\cdot, j] = \left(f_j(\omega^0), f_j(\omega^1), \ldots, f_j(\omega^{\ell-1})\right)^\top \tag{15}$$

The basis vectors $\{\mathbf{v}_i\}_{i=0}^{\ell-1}$ with $\mathbf{v}_i[j] = \omega^{ij}$ form a Vandermonde structure that ensures column-wise polynomial evaluation at powers of ω.

The security of this compression relies on the fundamental property that distinct column polynomials can collide at the evaluation points only if their difference vanishes at all these points simultaneously. This leads to the following collision resistance guarantee:

Lemma 1 (Compression Collision Resistance)**.** *For distinct matrices $\mathbf{M}_1, \mathbf{M}_2 \in \mathbb{F}_p^{n\times n}$ and uniformly sampled ω of order $\geq n$:*

$$\Pr_{\omega}\left[\Phi(\mathbf{M}_1) = \Phi(\mathbf{M}_2)\right] \leq \frac{n}{p} < 2^{-2\lambda}$$

Proof. Since $\mathbf{M}_1 \neq \mathbf{M}_2$, there exists at least one column j^* where the matrices differ. The difference polynomial $h(X) = f_{j^*}^{(1)}(X) - f_{j^*}^{(2)}(X)$ is non-zero with degree at most $n-1$. For a collision to occur, we need $h(\omega^i) = 0$ for all $i \in \{0, 1, \ldots, \ell-1\}$. Since h has at most $n-1$ roots and $\{\omega^i\}_{i=0}^{\ell-1}$ are distinct, the probability is bounded by n/p per Schwartz-Zippel. Under our parameter choice $p > 2^{2\lambda}n$, this probability is bounded by $2^{-2\lambda}$. □

The computational efficiency emerges from exploiting polynomial evaluation at roots of unity. When ω has order $N = 2^k \geq n$, each column compression requires computing ℓ evaluations $\{f_j(\omega^i)\}_{i=0}^{\ell-1}$ using a truncated Number-Theoretic Transform (NTT). The key insight is that evaluating $f_j(X)$ at the geometric sequence $\{1, \omega, \omega^2, \ldots, \omega^{\ell-1}\}$ corresponds to computing the first ℓ coefficients of the discrete Fourier transform, achievable via radix-2 algorithms in $O(n \log n)$ operations per column. This reduces total complexity from $O(n^3)$ to $O(n^2 \log n)$—for $n = 1024$, this achieves over $100\times$ speedup versus naive computation.

For KZG integration, the compressed matrix $\mathbf{M}_{(\text{comp})} \in \mathbb{F}_p^{\ell\times n}$ can be represented as a bivariate polynomial $F(X,Y) = \sum_{i=0}^{\ell-1}\sum_{j=0}^{n-1} \mathbf{M}_{(\text{comp})}[i,j]X^iY^j$. Since standard KZG commitments apply only to univariate polynomials, we use a two-step construction: for each row i, we define $F_i(Y) = \sum_{j=0}^{n-1} \mathbf{M}_{(\text{comp})}[i,j]Y^j$ and commit using an SRS of size n, obtaining $V_i = F_i(s)G_1$. These row commitments serve as coefficients of $G(X) = \sum_{i=0}^{\ell-1} V_i X^i$, producing the final commitment $V_{\text{outer}} = F(s,s)G_1$. This approach reduces the SRS size from $O(n^2)$ to $O(\max(n,\ell)) = O(n)$ group elements while maintaining collision probability below 2^{-128} for $\lambda = 128$. For concrete parameters with $n = 1024$, this reduces the SRS from $1,048,576$ to $1,024$ elements—a 99.9% reduction. The compression integrates transparently with the *zkMaP* protocol by operating on polynomial representations while preserving all verification properties, with the verifier remaining unaware of whether compression was used.

4.4 Verification

The verifier validates matrix multiplication claims through constant-time pairing operations derived from KZG commitments. Verification requires two phases: *Challenge*

Reconstruction: The verifier reconstructs the Fiat-Shamir challenge as:

$$y = \mathcal{H}(\text{pp} \parallel V_{\mathbf{A}} \parallel V_{\mathbf{B}} \parallel V_{\mathbf{C}}) \bmod p$$

where pp denotes public parameters. This ensures non-interactive soundness in the random oracle model.

Pairing Verification: The verifier checks: $e(V_W, sG_2 - yG_2) \stackrel{?}{=} e(V_\mu - P_{\mathbf{a}_y}(y) \cdot P_{\mathbf{b}_y}(y)G_1, G_2)$ Equivalently via bilinearity: $e(V_W, sG_2 - yG_2) \stackrel{?}{=} e(V_\mu - V_{\mathbf{a}_y} \cdot V_{\mathbf{b}_y}, G_2)$ This verifies the witness polynomial $W(x) = \frac{P_\mu(x) - P_{\mathbf{a}_y}(x) \cdot P_{\mathbf{b}_y}(x)}{x - y}$ encodes correct relationships. *Correctness*: For honest provers:

$$\begin{aligned} e(V_W, sG_2 - yG_2) &= e(W(s)G_1, sG_2 - yG_2) \\ &= e(\frac{P_\mu(s) - P_{\mathbf{a}_y}(s)P_{\mathbf{b}_y}(s)}{s - y}G_1, sG_2 - yG_2) \\ &= e((P_\mu(s) - P_{\mathbf{a}_y}(s)P_{\mathbf{b}_y}(s))G_1, G_2) \\ &= e(V_\mu - V_{\mathbf{a}_y} \cdot V_{\mathbf{b}_y}, G_2) \end{aligned}$$

Security: If $\mathbf{C} \neq \mathbf{A} \times \mathbf{B}$, Schwartz-Zippel bounds failure probability:

$$\Pr\left[P_\mu(y) = P_{\mathbf{a}_y}(y) \cdot P_{\mathbf{b}_y}(y)\right] \leq \frac{\deg}{p} < 2^{-128}$$

Soundness reduces to q-SDH under AGM. Verification completes in constant time ($\approx 3.6ms$) regardless of matrix size. Acceptance confirms $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ at the polynomial level, revealing no matrix entries. For a complete algorithmic description of the zkMaP protocol, see Algorithm 2.

5 Security Analysis

In this section, we present a comprehensive security analysis of *zkMaP*, establishing its security properties under the q-Strong Diffie–Hellman (q-SDH) assumption in the Algebraic Group Model (AGM) and Random Oracle Model (ROM). We provide explicit formalization of AGM extraction procedures, establish zero-knowledge guarantees across different interaction models, and demonstrate the relationship between our layered soundness theorems. We prove that *zkMaP* achieves perfect completeness, computational knowledge soundness, and zero-knowledge properties, making it a secure zero-knowledge proof system for matrix multiplication.

5.1 Security Model and Definitions

We establish formal definitions for *zkMaP*'s security properties:

Definition 5 (Secure Zero-Knowledge Proof System)**.** A zero-knowledge proof system for relation $\mathcal{R}$ is *secure* if it satisfies:

- *Perfect Completeness*: For any $(x, w) \in \mathcal{R}$, an honest prover $\mathcal{P}$ convinces verifier $\mathcal{V}$ with probability 1.
- *Computational Knowledge Soundness*: For any PPT adversary $\mathcal{A}$, there exists extractor $\mathcal{E}$ such that if $\mathcal{A}$ produces acceptable proof for x, then $\mathcal{E}$ extracts valid witness w with $(x, w) \in \mathcal{R}$, except with negligible probability.
- *Zero-Knowledge*: The proof reveals no information about w beyond $x \in \mathcal{L}_\mathcal{R}$. We distinguish:

– *Special Honest-Verifier ZK (SHVZK)*: Holds against honest verifiers using random challenges

– *Computational ZK in ROM*: Achieved via Fiat-Shamir against malicious verifiers

5.2 Layered Soundness Architecture

Our security analysis adopts a modular three-tier architecture, consistent with established methodologies in polynomial commitment-based zero-knowledge systems. This reflects the decomposition strategies of Sonic [MBKM19] (polynomial evaluation soundness before knowledge extraction), PLONK [GWC19] (permutation argument soundness before circuit satisfiability), and Marlin [CHM+20] (coefficient constraint soundness prior to global protocol soundness). The *zkMaP* hierarchy comprises: (i) polynomial commitment binding (Theorem 2), establishing that KZG commitments are computationally binding under q-SDH; (ii) witness polynomial soundness (Theorem 3), guaranteeing that accepting proofs satisfy $P_{\mathbf{C}}(s) = P_{\mathbf{A}}(s)P_{\mathbf{B}}(s)$; and (iii) knowledge soundness (Theorem 4), which constructs an AGM extractor using this consistency property.

This separation is technically essential for maintaining tight reductions. Theorem 3 ensures that, for any accepting proof, the difference polynomial $Q(x) = P_{\mathbf{C}}(x) - P_{\mathbf{A}}(x)P_{\mathbf{B}}(x)$ vanishes at the trapdoor point s, enabling the factorization $Q(x) = (x - s)R(x)$ used to extract a q-SDH solution in Theorem 4. This extraction-independent invariant avoids entangling polynomial consistency with the extraction procedure, preserving adversarial advantage ϵ and runtime $O(T)$—as opposed to the quadratic degradation ($\epsilon' = \epsilon/\text{poly}(n)$) that would result from a monolithic soundness proof. The modular structure improves clarity, supports independent verification of each layer, and follows best practices for composable cryptographic proofs.

5.3 Core Security Theorems

We now present the formal security theorems that establish *zkMaP*'s cryptographic guarantees. The theorems are organized according to their dependency relationships, with each building upon the security properties established by its predecessors.

Theorem 2 (Polynomial Commitment Binding)**.** *Under the q-Strong Diffie–Hellman (q-SDH) assumption, the polynomial commitments $V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}$ in the zkMaP protocol are computationally binding. Formally, for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ breaks the binding property is negligible:*

$$\Pr\begin{bmatrix}\mathsf{SRS} \leftarrow \mathsf{Setup}(1^\lambda, q), \\ (V, P_1, P_2) \leftarrow \mathcal{A}(\mathsf{SRS}) : \\ V = P_1(s)G_1 = P_2(s)G_1 \wedge P_1 \neq P_2\end{bmatrix} \leq \mathsf{negl}(\lambda),$$

where $\mathsf{SRS} = (G_1, sG_1, \ldots, s^qG_1, G_2, sG_2)$ *and* $\deg(P_1), \deg(P_2) \leq q$.

The binding property of polynomial commitments forms the foundation of our security analysis, ensuring that adversaries cannot equivocate about committed polynomials. This property is essential for the soundness of all subsequent reductions.

Theorem 3 (Witness Polynomial Soundness)**.** *Let λ be a security parameter and $q \in \mathbb{N}$. Assume the q-Strong Diffie–Hellman (q-SDH) assumption holds in the bilinear group setting. Then, the witness polynomial construction in the zkMaP protocol satisfies computational*

soundness. Specifically, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the probability that $\mathcal{A}$ produces a valid proof for an incorrect matrix multiplication is negligible:

$$\Pr\begin{bmatrix}\mathsf{SRS} \leftarrow \mathsf{Setup}(1^\lambda, q);\\ (\mathbf{A}, \mathbf{B}, \mathbf{C}, \tilde{\pi}) \leftarrow \mathcal{A}(\mathsf{SRS}) :\\ \mathsf{Verify}(\mathsf{SRS}, \mathbf{A}, \mathbf{B}, \mathbf{C}, \tilde{\pi}) = 1 \wedge \mathbf{C} \neq \mathbf{A} \times \mathbf{B}\end{bmatrix} \leq \mathsf{negl}(\lambda),$$

where $\tilde{\pi} = (V_{\mathbf{a}_y}, V_{\mathbf{b}_y}, V_\mu, V_W)$ is a forged proof tuple, and $\mathsf{SRS} = \left(G_1, sG_1, \ldots, s^q G_1, G_2, sG_2\right)$ is the structured reference string with secret $s \xleftarrow{\$} \mathbb{F}_p$.

Witness Polynomial Soundness serves as the crucial intermediate step that bridges the gap between commitment binding and full protocol soundness. This theorem guarantees that any proof accepted by the verifier must satisfy the polynomial relationship at the secret evaluation point, regardless of whether a valid witness can be extracted.

The proofs of Theorems 2 and 3 establish the foundational security properties required for our main result and are provided in Appendices A and B, respectively. Building upon these results, we now prove the complete security of the zkMaP protocol.

Theorem 4 (Completeness and Computational Knowledge Soundness of *zkMaP*)**.** *The zkMaP protocol satisfies:*

1. *Perfect Completeness: For honest prover $\mathcal{P}$ and verifier $\mathcal{V}$, if $\mathbf{C} = \mathbf{A} \times \mathbf{B}$, then $\Pr[\langle \mathcal{P}, \mathcal{V} \rangle = accept] = 1$.*
2. *Computational Knowledge Soundness: Under the q-SDH assumption in the AGM and ROM, for any PPT adversary $\mathcal{A}$, there exists an extractor $\mathcal{E}$ such that:*

$$\Pr\begin{bmatrix}\mathsf{SRS} \leftarrow \mathsf{Setup}(1^\lambda, n^2 - 1);\\ (V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) \leftarrow \mathcal{A}(\mathsf{SRS});\\ accept \leftarrow \mathcal{V}(\mathsf{SRS}, V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi);\\ (\mathbf{A}, \mathbf{B}, \mathbf{C}) \leftarrow \mathcal{E}^{\mathcal{A}}(\mathsf{SRS}) :\\ \mathbf{C} \neq \mathbf{A} \times \mathbf{B}\end{bmatrix} \leq \mathsf{negl}(\lambda).$$

Proof. Perfect Completeness: Let $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_p^{n \times n}$ be matrices with $\mathbf{C} = \mathbf{A} \times \mathbf{B}$. Their polynomial encodings satisfy $P_{\mathbf{C}}(x) = P_{\mathbf{A} \times \mathbf{B}}(x)$ by Theorem 2. The KZG commitments $V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}$ inherit perfect completeness when using an SRS of degree $d \geq n^2 - 1$.

Given Fiat-Shamir challenge $y \leftarrow \mathcal{H}(V_{\mathbf{A}} \parallel V_{\mathbf{B}}) \bmod p$ in the random oracle model, the projections $\mathbf{a}_y = \mathbf{y}_L^\top \mathbf{A}$ and $\mathbf{b}_y = \mathbf{B}\mathbf{y}_R$ satisfy $\mu = \mathbf{a}_y \cdot \mathbf{b}_y$ by matrix algebra. The witness polynomial given in Equation 14 has degree $\leq n^2 - 2$, supported by the SRS in Equation 10. Verification reduces to:

$$\begin{aligned} e(V_W, sG_2 - yG_2) &= e((s-y)W(s)G_1, G_2) \\ &= e((P_\mu(s) - P_{\mathbf{a}_y}(s)P_{\mathbf{b}_y}(s))G_1, G_2) \\ &= e(V_\mu - V_{\mathbf{a}_y} \cdot V_{\mathbf{b}_y}, G_2) \end{aligned}$$

Thus, the verification equation holds identically when $\mathbf{C} = \mathbf{A} \times \mathbf{B}$, proving perfect completeness.

Computational Knowledge Soundness: In the Algebraic Group Model (AGM), any PPT adversary $\mathcal{A}$ outputs group elements as linear combinations of SRS generators: In the

Algebraic Group Model (AGM), any PPT adversary $\mathcal{A}$ outputs group elements as linear combinations of SRS generators:

$$V_{\mathbf{A}} = \sum_{i=0}^{d} a_i(s^i G_1), \quad V_{\mathbf{B}} = \sum_{i=0}^{d} b_i(s^i G_1), \quad V_{\mathbf{C}} = \sum_{i=0}^{d} c_i(s^i G_1)$$

where $d = n^2 - 1$. Our extractor $\mathcal{E}$ (detailed in Algorithm 1) solves for coefficient vectors $\{a_i\}, \{b_i\}, \{c_i\}$ via Gaussian elimination over $\mathbb{F}_p$, then reconstructs matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ via Theorem 1's bijective encoding. By Witness Polynomial Soundness (Theorem 3), if verification passes but $\mathbf{C} \neq \mathbf{A} \times \mathbf{B}$, then the polynomial $Q(x) = P_{\mathbf{C}}(x) - P_{\mathbf{A}}(x)P_{\mathbf{B}}(x)$ satisfies $Q(s) = 0$ despite being nonzero. This forces the factorization $Q(x) = (x - s)R(x)$, enabling $\mathcal{E}$ to compute:

$$\frac{1}{s-y} G_1 = \frac{W(s)}{Q(y)} G_1$$

thereby solving the $(n^2 - 1)$-SDH problem, contradicting its assumed hardness. By the General Forking Lemma [BN06], if $\mathcal{A}$ succeeds with probability ϵ, then $\mathcal{E}$ extracts a valid witness with probability at least $\epsilon^2 - \mathsf{negl}(\lambda)$. By the Schwartz-Zippel lemma [Sch80], the probability of $Q(x)$ vanishing at a random point is at most $(2n^2 - 2)/p$, which is negligible for cryptographically large p. □

Algorithm 1 AGM Extractor for *zkMaP*

Require: Adversary $\mathcal{A}$, SRS $= \{s^i G_1\}_{i=0}^{d} \cup \{G_2, sG_2\}$ with $d = n^2 - 1$
Ensure: Extracted matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ or q-SDH solution

$(V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, \pi) \leftarrow \mathcal{A}(\text{SRS})$ ▷ $\pi = (V_{\mathbf{a}_y}, V_{\mathbf{b}_y}, V_\mu, V_W)$
Express $V_{\mathbf{A}} = \sum_{i=0}^{d} a_i(s^i G_1)$ ▷ Analogously for $V_{\mathbf{B}}, V_{\mathbf{C}}$
Solve linear system for $\{a_i\}, \{b_i\}, \{c_i\}$ via Gaussian elimination over $\mathbb{F}_p$
Reconstruct $\mathbf{A}$ via $P_{\mathbf{A}}(x) = \sum_{k=0}^{d} a_k x^k$ ▷ Bijective encoding
Reconstruct $\mathbf{B}$, $\mathbf{C}$ similarly from $\{b_i\}$, $\{c_i\}$
if $\mathbf{C} \neq \mathbf{AB}$ **then**
    Compute $Q(x) = P_{\mathbf{C}}(x) - P_{\mathbf{A}}(x)P_{\mathbf{B}}(x)$
    Factor $Q(x) = (x - s)R(x)$ using $Q(s) = 0$ ▷ By Thm 3
    Compute $y \leftarrow \mathcal{H}(V_{\mathbf{A}} \parallel V_{\mathbf{B}}) \bmod p$ ▷ Fiat-Shamir challenge
    Compute $\sigma \leftarrow (Q(y))^{-1} \cdot V_W$ ▷ Yields $\sigma = \frac{1}{s-y} G_1$
    return (y, σ) as q-SDH solution
else
    return $(\mathbf{A}, \mathbf{B}, \mathbf{C})$
end if

5.4 Algebraic Group Model Extraction

The computational knowledge soundness proof relies fundamentally on our ability to extract witnesses from successful adversaries in the Algebraic Group Model. In the AGM, all adversarial group elements must be algebraic combinations of the structured reference string (SRS). The *zkMaP* extractor leverages this to recover committed polynomials by solving a linear system over the SRS basis. Using standard techniques such as Gaussian elimination, the extractor reconstructs coefficient vectors for $P_{\mathbf{A}}(x)$, $P_{\mathbf{B}}(x)$, and $P_{\mathbf{C}}(x)$, and decodes the corresponding matrices via the bijective encoding. If $\mathbf{C} = \mathbf{AB}$, a valid witness is returned. Otherwise, the difference polynomial $Q(x) = P_{\mathbf{C}}(x) - P_{\mathbf{A}}(x)P_{\mathbf{B}}(x)$ is

nonzero and vanishes at s, enabling the factorization $Q(x) = (x - s)R(x)$. This enables computation of $\frac{1}{s-y}G_1$ from the committed witness polynomial, yielding a solution to the q-SDH problem. Algorithm 1 provides the complete extraction procedure with explicit steps for both witness recovery and q-SDH solution derivation. This extraction strategy follows the AGM framework of [FKL18] and avoids reliance on the trapdoor s, ensuring that knowledge soundness holds without requiring simulation trapdoors.

5.5 Zero-Knowledge Guarantees

Having established the soundness properties of *zkMaP*, we now turn to its zero-knowledge guarantees. The protocol achieves different levels of zero-knowledge depending on the interaction model, with perfect guarantees in the interactive setting and computational guarantees after applying the Fiat-Shamir transformation.

Theorem 5 (Zero-Knowledge Property of *zkMaP*)**.** *The zkMaP protocol satisfies* perfect special honest verifier zero-knowledge (SHVZK) *in the interactive setting under the q-SDH assumption. After applying the Fiat-Shamir transformation, it achieves computational zero-knowledge in the Random Oracle Model. For any verifier challenge $y \in \mathbb{F}_p$, there exists a probabilistic polynomial-time simulator $\mathcal{S}$ which, given only the public inputs $(V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}})$ and y, outputs a transcript statistically indistinguishable from a real protocol execution.*

Proof. Perfect SHVZK (Interactive Setting): We construct simulator $\mathcal{S}_{\text{SHVZK}}$ that on input public commitments $(V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}})$ and challenge y:

1. Samples $\mathbf{r}_a, \mathbf{r}_b \xleftarrow{\$} \mathbb{F}_p^n$ and defines projection polynomials:

$$\tilde{P}_{\mathbf{a}_y}(x) = \sum_{i=0}^{n-1} r_{a,i} x^i, \quad \tilde{P}_{\mathbf{b}_y}(x) = \sum_{i=0}^{n-1} r_{b,i} x^i$$

2. Samples random polynomial $\tilde{W}(x) \xleftarrow{\$} \mathbb{F}_p[x]$ with $\deg \leq 2n - 3$
3. Computes KZG commitments: $\tilde{V}_{\mathbf{a}_y} = \tilde{P}_{\mathbf{a}_y}(s)G_1, \quad \tilde{V}_{\mathbf{b}_y} = \tilde{P}_{\mathbf{b}_y}(s)G_1, \quad \tilde{V}_W = \tilde{W}(s)G_1$
4. Sets $\tilde{V}_\mu = \tilde{V}_{\mathbf{a}_y} \cdot \tilde{V}_{\mathbf{b}_y} + (s - y)\tilde{V}_W$

The transcript $(\tilde{V}_{\mathbf{a}_y}, \tilde{V}_{\mathbf{b}_y}, \tilde{V}_\mu, \tilde{V}_W)$ has identical distribution to real executions due to uniform polynomial coefficients and KZG's perfect hiding property. The simulator requires no witness or SRS trapdoor, establishing perfect SHVZK.

Computational ZK (Non-Interactive ROM): Following the Fiat-Shamir transformation [FS86] in the random oracle model [BR93], the simulator $\mathcal{S}_{\text{ROM}}$:

1. Programs $\mathcal{H}(V_{\mathbf{A}} \parallel V_{\mathbf{B}} \parallel V_{\mathbf{C}}) \to y$ for $y \xleftarrow{\$} \mathbb{F}_p$
2. Generates $\pi \leftarrow \mathcal{S}_{\text{SHVZK}}(V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, y)$
3. Maintains consistent random oracle responses

This achieves computational zero-knowledge against malicious verifiers under the random oracle assumption, without requiring SRS trapdoor access. This contrasts with trapdoor-dependent protocols like Groth16. □

5.6 Security Summary and Comparisons

The security analysis establishes that *zkMaP* achieves distinct zero-knowledge guarantees across interaction models. In interactive settings, the protocol provides perfect special honest-verifier zero-knowledge (SHVZK) through trapdoor-independent simulation: the simulator generates statistically indistinguishable transcripts without requiring SRS trapdoor access. For non-interactive setting via the Fiat-Shamir transformation [FS86] in the Random Oracle Model (ROM) [BR93], *zkMaP* achieves computational zero-knowledge against malicious verifiers. Here, the simulator programs the random oracle to ensure consistent challenge generation, with the *perfect* guarantee transitioning to *computational* due to hash function programmability requirements in ROM. Collectively, our theorems demonstrate that *zkMaP* satisfies the essential properties of a secure zero-knowledge proof system: perfect completeness where valid statements with honest proofs verify with probability 1, computational knowledge soundness enabling efficient witness extraction under q-SDH in AGM+ROM, and zero-knowledge providing perfect SHVZK interactively with computational ZK via Fiat-Shamir in ROM. These properties establish *zkMaP* as a cryptographically sound solution for privacy-preserving matrix multiplication verification.

Trusted Setup and Trapdoor Management: zkMaP's security model fundamentally differs from trapdoor-dependent protocols like Groth16. During trusted setup, trapdoor $s \in \mathbb{F}_p$ is generated via secure ceremony to compute $\mathsf{SRS} = \{s^i G_1\}_{i=0}^{n^2-1} \cup \{G_2, sG_2\}$, then immediately cryptographically erased. During operation, zero-knowledge simulators require no trapdoor access, relying instead on commitment randomness for interactive SHVZK and ROM programmability for non-interactive settings. The universal SRS supports multiple instances without per-execution setup, eliminating ongoing trapdoor trust while maintaining security.

Comparative Security Advantages: zkMaP provides distinctive security advantages over existing ZK protocols. Unlike Groth16's per-circuit trusted setup [Gro16], zkMaP employs a universal setup under the q-SDH assumption with AGM analysis. Compared to zkMatrix [CYY24], *zkMaP* leverages q-SDH and AGM foundations rather than RSA assumptions, providing explicit extraction procedures with concrete security guarantees. Relative to QuickSilver [YSWW21], our protocol operates in bilinear groups under structured assumptions while maintaining constant-size proofs, offering different security-efficiency trade-offs suitable for matrix-specific applications.

The protocol's layered soundness architecture enables compositional security proofs while avoiding circuit overhead through matrix-specific optimizations. While achieving SHVZK in interactive settings, the Fiat-Shamir transformation yields computational ZK security comparable to state-of-the-art systems, establishing *zkMaP* as a robust and practically viable solution for zero-knowledge matrix multiplication.

6 Batch Verification

The *zkMaP* protocol is designed to support the simultaneous verification of multiple matrix multiplication proofs while preserving its zero-knowledge properties. To achieve this, our scheme employs a batching mechanism that aggregates individual proofs into a single, compact proof. This design not only reduces the communication and computational overhead associated with verifying each instance separately but also maintains the algebraic structure required for sound verification.

For a collection of t matrix multiplication instances $\{(\mathbf{A}_i, \mathbf{B}_i, \mathbf{C}_i)\}_{i=1}^{t}$ with $\mathbf{C}_i = \mathbf{A}_i \times \mathbf{B}_i$, each instance is first processed individually. The matrices are encoded into polynomials and committed using KZG commitments, yielding commitments $V_{\mathbf{A}_i}$, $V_{\mathbf{B}_i}$, and $V_{\mathbf{C}_i}$ for each instance. Furthermore, for each individual proof, a witness polynomial is constructed

and committed to, resulting in witness commitments V_{W_i}.

To aggregate these proofs, our protocol employs a standard random linear combination technique. A random scalar ρ is sampled uniformly from $\mathbb{F}_p$; such a choice is a well-established practice in the literature (e.g., in batch verification schemes for SNARKs and polynomial commitment protocols) as it prevents an adversary from crafting false proofs that cancel out in the aggregation. Since the probability of sampling zero challenges is negligible ($1/p \approx 2^{-256}$), the protocol rejects this case and resamples to ensure well-defined witness polynomials. The individual commitments are then aggregated via weighted sums:

$$V_{\mathbf{A}}^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} V_{\mathbf{A}_i}, \quad V_{\mathbf{B}}^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} V_{\mathbf{B}_i}, \quad V_{\mathbf{C}}^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} V_{\mathbf{C}_i}.$$

Similarly, the batch witness commitment is computed as: $V_W^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} V_{W_i}$.

The use of the powers of ρ ensures that the contributions of the individual proofs remain distinct in the aggregate, preserving the necessary polynomial relationships for sound verification. After the commitments have been aggregated, the final verification is performed by checking a pairing equation:

$$e\Big(V_W^{\text{batch}}, sG_2 - y^{\text{batch}} G_2\Big) = e\Big(V_{\mathbf{C}}^{\text{batch}}, G_2\Big) \cdot e\Big(V_{\mathbf{A}}^{\text{batch}}, G_2\Big)^{-1} \cdot e\Big(V_{\mathbf{B}}^{\text{batch}}, G_2\Big)^{-1}$$

where the challenge y^{batch} is obtained by similarly aggregating the individual challenges y_i from each proof. The pairing check is executed using the ate pairing, and thanks to optimized multi-pairing techniques, the effective cost is reduced to only two pairing computations, regardless of the number of batched instances.

The security of the batch scheme is underpinned by the randomness of ρ, which guarantees that any adversarial attempt to forge or cancel out individual proofs is detected with high probability. Specifically, the probability of an invalid proof being accepted is bounded by $\frac{d}{p}$, where d is the maximum degree of the underlying polynomials and p is the field size. This bound is standard in polynomial commitment schemes and ensures that the overall batch verification remains both efficient and secure.

Theorem 6 (Batch *zkMaP* Protocol)**.** *Let $(\mathbf{A}_i, \mathbf{B}_i, \mathbf{C}_i)$ for $i \in [1, t]$ be matrix multiplication instances over $\mathbb{F}_p^{n \times n}$. The batch zero-knowledge matrix product (zkMaP) protocol satisfies:*

1. ***Perfect Completeness**: If $\mathbf{C}_i = \mathbf{A}_i \times \mathbf{B}_i$ for all i, then the verifier always accepts.*
2. ***Computational Zero-Knowledge in ROM**: There exists a PPT simulator producing proofs computationally indistinguishable from honest ones in the random oracle model*
3. ***Computational Soundness**: If a malicious prover produces an accepting proof where $\exists j$ such that $\mathbf{C}_j \neq \mathbf{A}_j \times \mathbf{B}_j$, the q-SDH assumption can be broken.*

A proof of this theorem is provided in Appendix C, building on the AGM extraction framework (Algorithm 1) and zero-knowledge simulation strategies established in Section 5.

For further implementation details and a complete procedural description, refer to Algorithm 3, which outlines the steps for setting up the SRS, generating individual proofs, aggregating commitments via the random challenge ρ, and performing the batch verification.

7 Complexity Analysis of the *zkMaP* Protocol

7.1 Computational Complexity

We analyze the computational complexity of the *zkMaP* protocol for verifying the relation $\mathbf{C} = \mathbf{A} \times \mathbf{B}$ in zero-knowledge, where $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_p^{n \times n}$. We assume standard costs: field multiplication C_{mul}, field addition C_{add}, exponentiation in the group C_{exp}, pairing evaluation C_{pair}, hashing for Fiat-Shamir instantiation C_{hash}, and small-exponent powering C_{pow}.

Setup Complexity: The trusted setup generates the structured reference string (SRS). For uncompressed matrices, we require SRS $= \{G_1, sG_1, \ldots, s^{n^2-1}G_1, G_2, sG_2\}$, costing $(n^2 + 1) \cdot C_{\text{exp}}$. With Vandermonde compression (Section 4.3), the SRS reduces to $O(\max(n, \ell))$ elements where $\ell = \max(\lceil n/4 \rceil, 2\lambda)$, substantially reducing setup costs for large matrices. Committing to matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ requires encoding them into polynomials and evaluating at point s, costing $3n^2(C_{\text{mul}} + C_{\text{add}}) + 3n^2 \cdot C_{\text{exp}}$ for the uncompressed case. With compression, this reduces to $O(\ell n)$ operations per matrix. Thus, the setup phase totals:

$$T_{\text{Setup}} = (n^2 + 1)C_{\text{exp}} + 3n^2(C_{\text{mul}} + C_{\text{add}} + C_{\text{exp}}).$$

***Challenge Generation*:** The verifier computes the Fiat-Shamir challenge as $y = \mathcal{H}(\text{pp} \| V_{\mathbf{A}} \| V_{\mathbf{B}} \| V_{\mathbf{C}}) \bmod p$, where pp denotes public parameters, costing C_{hash}. The prover derives structured challenge vectors:

$$\mathbf{y}_L = (1, y^n, y^{2n}, \ldots, y^{(n-1)n}), \quad \mathbf{y}_R = (1, y, y^2, \ldots, y^{n-1}),$$

requiring $2n \cdot C_{\text{pow}}$. The challenge generation phase thus costs: $T_{\text{Challenge}} = C_{\text{hash}} + 2n \cdot C_{\text{pow}}$.

***Prover Computation*:** The prover computes projections $\mathbf{a}_y = \mathbf{y}_L^\top \mathbf{A}$, $\mathbf{b}_y = \mathbf{B}\mathbf{y}_R$, and $\mu = \mathbf{y}_L^\top \mathbf{C}\mathbf{y}_R$. Each projection involves approximately n^2 multiplications and n^2 additions. The witness polynomial:

$$W(x) = \frac{P_\mu(x) - P_{\mathbf{a}_y}(x) \cdot P_{\mathbf{b}_y}(x)}{x - y}$$

requires polynomial operations costing approximately n^2 field operations. Collectively, the prover complexity is:

$$T_{\text{Prover}} \approx 5n^2(C_{\text{mul}} + C_{\text{add}}) + 3n^2 C_{\text{exp}} + O(n).$$

***Verification*:** The verifier checks the pairing equation:

$$e(V_W, sG_2 - yG_2) \stackrel{?}{=} e(V_\mu - V_{\mathbf{a}_y} \cdot V_{\mathbf{b}_y}, G_2).$$

This requires exactly two pairing computations via multi-pairing optimization, achieving constant verification time (3.7 ms in our benchmarks) regardless of matrix dimensions. Hence, verification complexity is:

$$T_{\text{Verify}} = 2 \cdot C_{\text{pair}} + n(C_{\text{mul}} + C_{\text{add}}).$$

***Batch Verification*:** For t matrix multiplication instances, our batch protocol aggregates proofs using random linear combinations. The batch aggregation computes $y^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} y_i$, where ρ is a random challenge and y_i are individual proof challenges. This introduces $O(t)$ field operations, while the constant two pairings are preserved regardless of t. The complete batch verification complexity is

$$T_{\text{Verify-Batch}} = 2 \cdot C_{\text{pair}} + t(C_{\text{mul}} + C_{\text{add}}),$$

where t denotes the batch size. Experimental evidence indicates that, for practical matrix dimensions, the time spent on field operations during batch verification is negligible compared to the time required for the constant two pairing evaluations. For instance, with $t = 1024$, all field operations taking less than 0.1 ms in total, whereas a single pairing operation requires approximately 1.6 ms, confirming that zkMaP verification is pairing-dominated and consistent with standard cryptographic practice.

***Comparative Operation-Level Analysis*:** To contextualize zkMaP's efficiency gains, we contrast its verification costs with those of zkMatrix [CYY24]. For a single proof, zkMatrix requires $4C_{\text{pair}} + (14\log_2 n + 38)C_{\text{exp}} + (3\log_2 n + 9)C_{\text{hash}}$, while batch verification of t proofs costs $4C_{\text{pair}} + ((2t+12)\log_2 n + 13t + 25)C_{\text{exp}}$. In comparison, zkMaP achieves the same task with only $2C_{\text{pair}} + n(C_{\text{mul}} + C_{\text{add}})$ for single proofs and $2C_{\text{pair}} + t(C_{\text{mul}} + C_{\text{add}})$ for batch verification. This represents a 50% reduction in pairing operations and the replacement of costly logarithmic-scale exponentiations with lightweight field multiplications. Since exponentiations scale as $\log_2 p$ field multiplications and pairings dominate verification time, zkMaP's constant two-pairing design delivers substantial asymptotic and concrete performance advantages, particularly for large matrices and high-throughput batch settings.

7.2 Memory Complexity

The memory complexity of zkMaP is analyzed for both single-proof and batch-proof scenarios, with emphasis on scalability in large-scale matrix multiplication verification.

Single Proof: For a single proof, the prover's memory is dominated by the storage of three $n \times n$ matrices (i.e., $3n^2$ field elements), polynomial commitment coefficients, and intermediate computation state, yielding overall $O(n^2)$ memory complexity. With Vandermonde compression enabled, storage reduces to $O(\ell n)$ per matrix where $\ell = \max(\lceil n/4 \rceil, 2\lambda)$, providing substantial memory savings for large matrices. The total memory usage is:

$$M_{\text{single}} = O(n^2) \cdot (C_{\text{mem}} + C_{\text{poly}})$$

where C_{mem} accounts for matrix storage and C_{poly} captures polynomial commitment coefficients.

Batch Proofs: Naive verification of t proofs requires $O(tn^2)$ memory, which is prohibitive for large t. zkMaP mitigates this via *sub-batching*, processing at most b proofs concurrently. Peak memory becomes

$$M_{\text{batch}} = O(bn^2) \cdot (C_{\text{mem}} + C_{\text{poly}})$$

while the prover runtime is

$$T_{\text{prover}}^{\text{batch}} = \left\lceil \frac{t}{b} \right\rceil \left[5bn^2(C_{\text{mul}} + C_{\text{add}}) + 3bn^2 C_{\text{exp}} \right]$$

Verification memory remains constant at C_{pair} due to proof aggregation.

Trade-Off Characterization: Sub-batching reduces peak memory from tn^2 to bn^2 while incurring a time overhead proportional to t/b. The trade-off relationship is characterized by Memory Reduction Factor $= t/b$ and Time Overhead Factor $= \lceil t/b \rceil$. In our implementation, setting $b = 8$ reduces memory usage to 12.5% of naive batching with only a 12.5% increase in prover time.

Parameter Selection: The maximum admissible sub-batch size b is bounded by the available system memory $M_{\text{available}}$ as:

$$b \leq \left\lfloor \frac{M_{\text{available}}}{n^2 \cdot (C_{\text{mem}} + C_{\text{poly}})} \right\rfloor$$

This tunable parameter allows zkMaP to scale from memory-constrained embedded devices to high-performance servers, providing practitioners with explicit guidelines to balance memory and performance.

Table 2: Computational and memory complexity of zkMaP protocol phases.

Phase	**Computational Complexity**	**Memory Complexity**
Setup	$(n^2+1)C_{\text{exp}} + 3n^2(C_{\text{mul}} + C_{\text{add}} + C_{\text{exp}})$	$n^2 C_{\text{mem}}$
Setup (Compressed)	$O(\max(n,\ell))C_{\text{exp}} + 3\ell n(C_{\text{mul}} + C_{\text{add}} + C_{\text{exp}})$	$\ell n C_{\text{mem}}$
Challenge Generation	$C_{\text{hash}} + 2nC_{\text{pow}}$	C_{hash}
Prover (Single)	$5n^2(C_{\text{mul}} + C_{\text{add}}) + 3n^2 C_{\text{exp}} + O(n)$	$n^2(C_{\text{mem}} + C_{\text{poly}})$
Prover (Batch)	$\lceil \frac{t}{b} \rceil \left[5bn^2(C_{\text{mul}} + C_{\text{add}}) + 3bn^2 C_{\text{exp}}\right]$	$bn^2(C_{\text{mem}} + C_{\text{poly}})$
Verification (Single)	$2C_{\text{pair}} + n(C_{\text{mul}} + C_{\text{add}})$	C_{pair}
Verification (Batch)	$2C_{\text{pair}} + t(C_{\text{mul}} + C_{\text{add}})$	C_{pair}

8 Implementation and Results

8.1 Implementation Architecture

Our implementation uses the Kate-Zaverucha-Goldberg (KZG) polynomial commitment scheme as its foundation. The system is implemented in Rust, leveraging the Arkworks cryptographic library with the BLS12-381 pairing-friendly elliptic curve, which provides 128 bits of security with a 255-bit scalar field. The core modules include a `KZG` implementation for polynomial commitments, a `ZKMatrixProof` structure for zero-knowledge matrix multiplication proofs, and an `OptimizedBatchedZKMatrixProof` structure that implements an efficient batching protocol.

We implemented several technical optimizations to enhance practical efficiency: parallel processing via Rayon's work-stealing scheduler to accelerate matrix-to-polynomial conversion; memory-efficient matrix flattening techniques that significantly reduce overhead; and challenge-based proof aggregation using Fiat-Shamir transformations for non-interactive proofs. These optimizations collectively contribute to the performance characteristics observed in our experimental evaluation. To support artifact review and ensure reproducibility, we provide an anonymized GitHub repository containing the full implementation, benchmarking scripts, and documentation. The repository is accessible at: https://github.com/20code-submission-review25/anonzkp-harbor-6092.

8.2 Experimental Setup

Experiments were conducted on an HP Z840 Workstation equipped with an Intel Xeon E5-2680 v3 processor (48 cores at 2.50GHz) and 128GB RAM, running Ubuntu 22.04.3 LTS.

We measured proving time, verification time, proof size, and memory usage across varying matrix dimensions from 8×8 to 1024×1024. For batch proof performance, we measured the time required to generate and verify multiple proofs simultaneously, with batch sizes ranging from 4 to 1024. To ensure accurate memory measurements, we implemented a custom global allocator that tracks peak memory usage during proof generation.

8.3 Performance Analysis

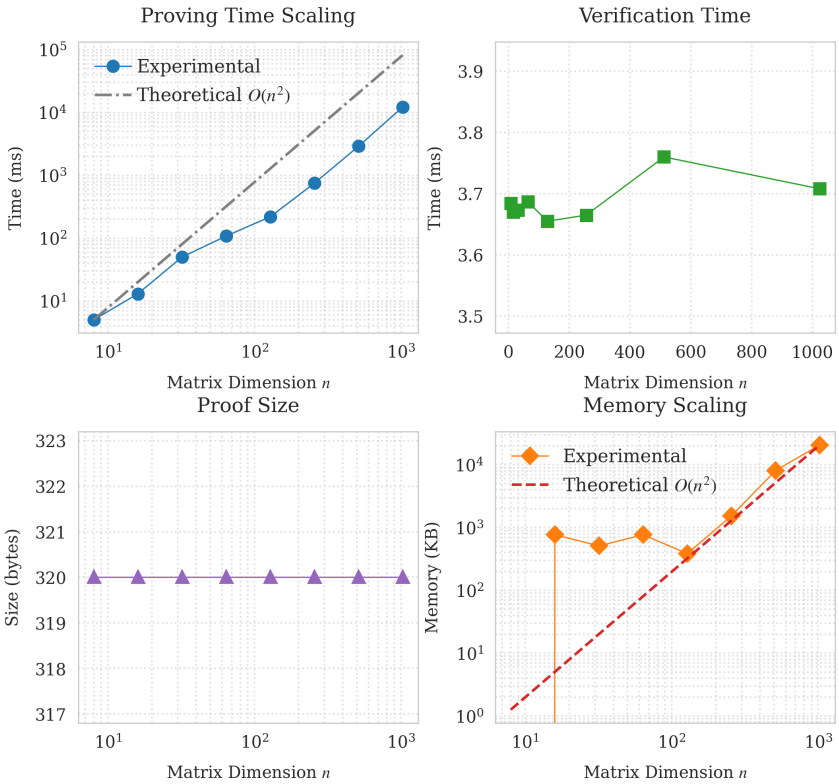

Figure 1: Zero-Knowledge Matrix Multiplication Performance. Top left: Proving time vs. matrix size. Top right: Verification time remains constant regardless of matrix size. Bottom left: Proof size remains fixed at 320 bytes. Bottom right: Memory usage scales efficiently with matrix size.

Our implementation demonstrates remarkable scaling properties for matrix multiplication proofs. As shown in Figure 1, the proving time exhibits an initial increase from 5 ms for 8×8 matrices to 50 ms for 32×32 matrices, followed by gradual growth, reaching 12,211 ms (approximately 12.21 s) for 1024×1024 matrices. This behavior aligns with the expected $O(n^2)$ complexity of our proof generation process.

The verification time remains nearly constant at around 3.68 ms across all matrix dimensions, demonstrating the protocol's ability to verify matrix multiplications in constant time regardless of input size. Similarly, the proof size is fixed at 320 bytes for all matrix dimensions, indicating a communication complexity that is independent of matrix size. Memory usage scales efficiently, increasing from negligible usage for very small matrices to 20,672 KB (approximately 20.2 MB) for 1024×1024 matrices, which corroborates our theoretical analysis of $O(n^2)$ memory complexity.

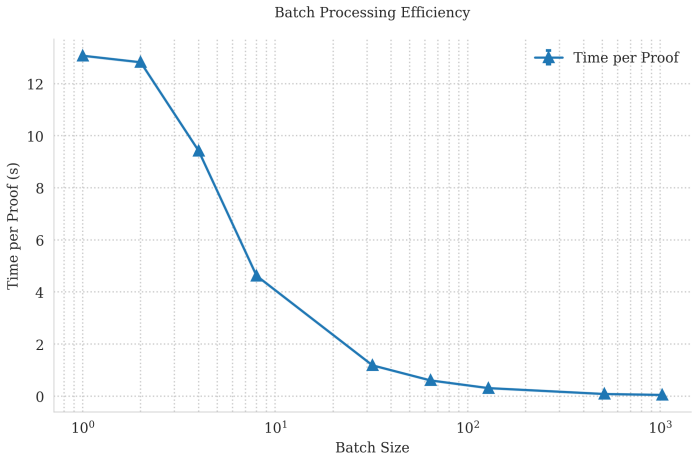

Figure 2: Batching efficiency of our protocol: average time per proof as a function of batch size, using a sub-batching parameter of $b = 8$.

Our batching protocol demonstrates significant efficiency gains in multi-proof scenarios. As illustrated in Figure 2, the average time per proof decreases dramatically from 12.37 s for individual proofs to approximately 46.79 ms at a batch size of 1,024, representing a 279× speedup. The protocol retains a constant verification time of approximately 3.71 ms and fixed proof size of 320 bytes, regardless of the batch size.

Parallelization significantly accelerates proof generation, particularly for small to medium-sized matrices. As shown in Table 4 in the Appendix D, the proving time for 256×256 matrices decreases from 16.02 s on a single thread to 0.90 s on 48 threads, achieving a 17.78× speedup. Similarly, 512×512 matrices see a reduction from 66.14 s to 3.02 s, yielding a 21.92× improvement. However, for larger matrices such as 1024×1024, the benefits of multithreading are curtailed by memory bandwidth and synchronization overheads, with proving time decreasing from 287.39 s to 12.50 s—a 22.99× speedup. These results underscore that while multithreading is highly effective for moderate workloads, alternative strategies like batching may be necessary to further enhance scalability for larger inputs.

8.4 Comparison with Prior Work

Table 3: Comparison of performance metrics for 1024×1024 matrix multiplication

Protocol	Transcript Size	Prover RAM Usage	Prover Time	Verifier Time
zkMatrix (Single)	7.67KB	176MB	197.04s	70.15ms
zkMatrix (Batch, total)	1.42MB	304MB	512.48s	12.06s
zkMatrix (Batch, per proof)	1.42KB	304KB	0.50s	11.77ms
Our Work (Single)	**320B**	**24MB**	**12.21s**	**3.71ms**
Our Work (Batch, total)	**320B**	**197MB**	**47.91s**	**3.73ms**
Our Work (Batch, per proof)	**320B**	**192KB**	**46.79ms**	**3.73ms**
QuickSilver	25.2MB	1GB	10.0s	N/A
Spartan	≤100KB	600GB	≥5000s	N/A
Virgo	221KB	148GB	357s	N/A

Table 3 presents a performance comparison of our approach against state-of-the-art systems, with a primary focus on zkMatrix as the closest prior work. The reported values for zkMatrix and the other protocols are taken from their published results, ensuring a

consistent benchmark environment.

Our verification time remains constant at 3.68 ms, irrespective of batch size, due to its aggregation-based strategy with $O(1)$ complexity. This yields a 19.07× improvement over zkMatrix's single-proof verification time of 70.15 ms. For prover efficiency, our scheme achieves a 16.14× speedup, reducing the proving time for a single 1024×1024 matrix multiplication from 197.04 s (zkMatrix) to 12.21 s. In batched proof generation, zkMatrix requires 512.48 s for processing, while our approach completes the same task in 47.9 s, reflecting a 10.7× reduction in total prover time. On a per-proof basis, our scheme achieves a 10.68× speedup, reducing zkMatrix's 500 ms proving time per proof to 46.79 ms. Unlike zkMatrix, where verification time scales with batch size (increasing from 70.15 ms to 12.06 s for 1024 proofs), our approach maintains a constant verification time of 3.68 ms, ensuring scalability.

When comparing our approach with QuickSilver, several technical differences explain the performance characteristics. We use KZG commitments based on elliptic curve pairings, which enable constant-size proofs but require a trusted setup. QuickSilver uses a subfield VOLE (Vector Oblivious Linear Evaluation) based approach, which doesn't require a trusted setup but results in larger proof sizes. Our approach leverages efficient matrix-to-polynomial conversion using techniques optimized for the KZG commitment scheme, while QuickSilver represents matrix multiplication as a collection of inner products. Our batching strategy uses challenge-based aggregation to achieve a 45.5× speedup, while QuickSilver focuses on reducing communication complexity rather than batching efficiency.

Both approaches offer robust security guarantees. Our implementation provides computational security based on the hardness of the discrete logarithm problem in elliptic curve groups, with the BLS12-381 curve offering 128-bit security, whereas QuickSilver provides 128-bit computational security with statistical security parameters that vary with field size.

8.5 Practical Implications

A straightforward method for verifying matrix multiplication requires transmitting full input matrices and the resulting product, leading to a communication complexity of $O(n^2)$. For example, verifying 1024×1024 matrices with 32-byte field elements would involve approximately 3GB of data transfer per proof. In addition, traditional verification requires $O(n^3)$ computations, amounting to over one billion field multiplications for matrices of this size, which is impractical in many real-world scenarios.

In contrast, our approach drastically reduces both communication and computational overhead. By achieving $O(1)$ communication and verification time, our protocol is particularly well-suited for blockchain and distributed ledger systems where minimizing on-chain storage and verification costs is crucial. Specifically, our protocol attains a constant verification time of approximately 3.71 ms and produces a compact proof of only 320 bytes, representing a significant reduction in communication compared to naive methods.

Figure 3 illustrates the performance comparison between standard (non-zero-knowledge) and our *zkMaP*-based matrix multiplication approaches. The y-axis is presented on a logarithmic scale to highlight relative performance trends across various matrix sizes. while non-ZK computation shows cubic growth $O(n^3)$, requiring 102.79 s at $n = 1024$, *zkMaP* proving scales quadratically $O(n^2)$, needing only 12.21 s. Verification remains constant-time $O(1)$ at 3.71 ms regardless of matrix size. At $n = 256$, *zkMaP* achieves a 1.46× speedup over standard computation, increasing to 8.4× at $n = 1024$. These results confirm our theoretical analysis: $O(n^2)$ for proving and $O(1)$ for verification, outperforming conventional $O(n^3)$ methods. The proof size remains fixed at 320 bytes.

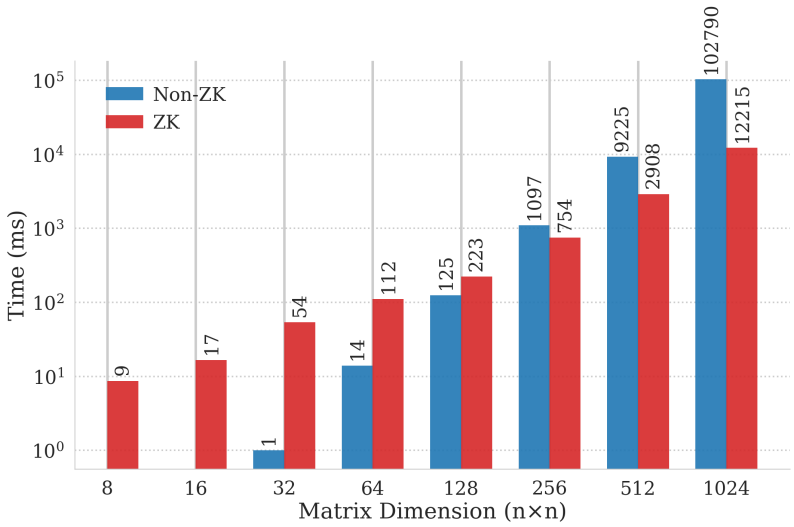

Figure 3: Performance comparison between standard (non-ZK) and *zkMaP*-based matrix multiplication on a logarithmic scale (milliseconds). At $n = 1024$, *zkMaP* demonstrates competitive performance, with a modest overhead relative to the standard method, thus establishing its practical viability in real-world applications.

Overall, the substantial improvements in both theoretical asymptotic complexity and practical performance metrics underscore the viability of our method for efficient zero-knowledge proofs in large-scale matrix multiplication applications.

8.6 Real-World Deployment Considerations

zkMaP's efficiency and fixed-size proofs enable deployment across a wide range of real-world systems. On blockchain platforms such as Ethereum, the 320-byte proof and deterministic two-pairing verification ensure predictable gas costs, while the non-interactive protocol reduces transaction rounds, latency, and on-chain expenses. For resource-constrained devices, including IoT and edge nodes, configurable sub-batching allows proofs to be processed even with limited memory, and the constant verification footprint guarantees predictable resource usage regardless of proof complexity. Furthermore, the fixed-size proof format and standard binary serialization facilitate seamless integration with virtual machines and cryptographic libraries, eliminating dynamic memory allocation and ensuring compatibility across programming environments and network protocols. These properties collectively support deployment in privacy-preserving frameworks such as decentralized identity verification, secure multi-party computation, and confidential smart contract execution.

9 Conclusion

We have presented *zkMaP*, a novel zero-knowledge proof protocol for matrix multiplication achieves high efficiency through polynomial encodings, KZG commitments, and structured inner-product reductions. By reducing verification to a single constant-size pairing equation, *zkMaP* requires only two pairing operations regardless of matrix dimensions, achieves

optimal $O(n^2)$ prover complexity, and generates compact proofs of 320 bytes—representing a 96% reduction compared to prior schemes.

Our experimental evaluation demonstrates significant practical improvements: For 1024×1024 matrices, *zkMaP* achieves a 16.14× speedup over zkMatrix with proof generation in 12.21 seconds using just 24.6 MB memory. The protocol scales efficiently in batch processing scenarios, achieving per-proof times of 46.79 ms while maintaining constant verification at 3.71 ms. These results position *zkMaP* as a practical solution for real-world applications requiring privacy-preserving machine learning, secure analytics, and collaborative computation.

While *zkMaP* currently relies on the q-SDH assumption in bilinear groups and is not post-quantum secure, future work will explore lattice-based alternatives such as SLAP [AFLN24] to achieve quantum resistance. Additional research directions include extensions to tensor operations, integration with federated learning frameworks, and GPU acceleration for further performance gains.

References

[AFLN24] Martin R. Albrecht, Giacomo Fenzi, Oleksandra Lapiha, and Ngoc Khanh Nguyen. Slap: Succinct lattice-based polynomial commitments from standard assumptions. In *Advances in Cryptology – EUROCRYPT 2024: 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26–30, 2024, Proceedings, Part VII*, volume 14657 of *LNCS*, pages 90–119. Springer, 2024. doi:10.1007/978-3-031-58754-2_4.

[BBB+18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE, 2018. doi:10.1109/SP.2018.00020.

[BCCT12] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *ITCS '12: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, pages 326–349. ACM, 2012. doi:10.1145/2090236.2090263.

[BGM17] Sean Bowe, Ariel Gabizon, and Ian Miers. Scalable multi-party computation for zk-snark parameters in the random beacon model. *Cryptology ePrint Archive*, 2017. URL: https://eprint.iacr.org/2017/1050.

[BLS04] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. *Journal of Cryptology*, 17(4):297–319, 2004. doi:10.1007/s00145-004-0314-9.

[BN06] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *CCS '06: Proceedings of the 13th ACM Conference on Computer and Communications Security*, pages 390–399. ACM, 2006. doi:10.1145/1180405.1180453.

[BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *CCS '93: Proceedings of the 1st ACM Conference on Computer and Communications Security*, pages 62–73. ACM, 1993. doi:10.1145/168588.168596.

[BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. URL: https://eprint.iacr.org/2018/046.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Advances in Cryptology – EUROCRYPT 2023: 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23–27, 2023, Proceedings, Part II*, volume 14077 of *LNCS*, pages 499–530. Springer, 2023. doi:10.1007/978-3-031-30617-4_17.

[CFQ19] Matteo Campanelli, Dario Fiore, and Anaïs Querol. LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In *CCS '19: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2075–2092. ACM, 2019. doi:10.1145/3319535.3339820.

[CHM+20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zksnarks with universal and updatable srs. In *Advances in Cryptology – EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, 2020. doi:10.1007/978-3-030-45721-1_26.

[CYY24] Mingshu Cong, Tsz Hon Yuen, and Siu-Ming Yiu. zkMatrix: Batched short proof for committed matrix multiplication. In *AsiaCCS '24: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pages 289–305. ACM, 2024. doi:10.1145/3634737.3645003.

[FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In *Advances in Cryptology – CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II*, volume 10991 of *LNCS*, pages 33–62. Springer, 2018. doi:10.1007/978-3-319-96881-0_2.

[FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO 1986: 6th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 1986, Proceedings*, volume 263 of *LNCS*, pages 186–194. Springer, 1986. doi:10.1007/3-540-47721-7_12.

[GO94] Oded Goldreich and Yair Oren. Definitions and properties of zero-knowledge proof systems. *Journal of Cryptology*, 7(1):1–32, 1994. doi:10.1007/BF00195207.

[Gro09] Jens Groth. Linear algebra with sub-linear zero-knowledge arguments. In *Advances in Cryptology – CRYPTO 2009: 29th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 16–20, 2009, Proceedings*, volume 5677 of *LNCS*, pages 192–208. Springer, 2009. doi:10.1007/978-3-642-03356-8_12.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, 2016. doi:10.1007/978-3-662-49896-5_11.

[GWC19] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: https://eprint.iacr.org/2019/953.

[KZG10] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology – ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010, Proceedings*, volume 6476 of *LNCS*, pages 177–194. Springer, 2010. doi:10.1007/978-3-642-17373-8_11.

[MBKM19] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. Sonic: Zero-knowledge snarks from linear-size universal and updatable structured reference strings. In *CCS '19: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2111–2128. ACM, 2019. doi:10.1145/3319535.3339817.

[PHGR16] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly Practical Verifiable Computation. *Communications of the ACM*, 59(2):103–112, 2016. doi:10.1145/2856449.

[Sch80] Jacob T Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)*, 27(4):701–717, 1980. doi:10.1145/322217.322225.

[Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In *Advances in Cryptology – CRYPTO 2013: 33rd Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2013, Proceedings*, pages 71–89. Springer, 2013. doi:10.1007/978-3-642-40084-1_5.

[WTS+18] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy*, pages 926–943. IEEE, 2018. doi:10.1109/SP.2018.00060.

[XZZ+19] Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology – CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III*, volume 11693 of *LNCS*, pages 733–764. Springer, 2019. doi:10.1007/978-3-030-26954-8_24.

[YSWW21] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *CCS '21: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2986–3001. ACM, 2021. doi:10.1145/3460120.3484556.

A Proof of Theorem 2

Proof. Assume for contradiction that there exists a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ that breaks the binding property of KZG polynomial commitments with non-negligible advantage $\epsilon(\lambda)$, where λ is the security parameter. We construct a reduction $\mathcal{B}$ that uses $\mathcal{A}$ as a subroutine to solve the q-Strong Diffie-Hellman (q-SDH) problem with the same advantage.

Reduction Setup. Given a q-SDH challenge instance $\left(G_1, sG_1, s^2G_1, \ldots, s^qG_1, G_2, sG_2\right)$ where $s \xleftarrow{\$} \mathbb{F}_p$ is unknown, the reduction $\mathcal{B}$ constructs the structured reference string $\mathsf{SRS} = (G_1, sG_1, \ldots, s^qG_1)$ and provides it to $\mathcal{A}$.

Adversarial Output and Collision Analysis. Suppose $\mathcal{A}$ outputs a binding collision (V, P_1, P_2) where $P_1(x), P_2(x) \in \mathbb{F}_p[x]$ are distinct polynomials with $\deg(P_1), \deg(P_2) \leq q$, yet $P_1(s)G_1 = P_2(s)G_1 = V$.

Define the difference polynomial $Q(x) = P_1(x) - P_2(x) \in \mathbb{F}_p[x]$. Since $P_1 \neq P_2$, we have $Q(x) \not\equiv 0$ with $\deg(Q) \leq q$. From the collision:

$$Q(s)G_1 = (P_1(s) - P_2(s))G_1 = V - V = 0_{G_1}$$

Since G_1 has prime order p, this implies $Q(s) = 0$.

Polynomial Factorization and Extraction. By the Factor Theorem, there exists a unique polynomial $R(x) \in \mathbb{F}_p[x]$ such that:

$$Q(x) = (x - s)R(x)$$

with $\deg(R) = \deg(Q) - 1 \leq q - 1$. The reduction $\mathcal{B}$ computes $R(x)$ via polynomial division of $Q(x)$ by $(x - s)$.

Construction of q-SDH Solution. Using the SRS, $\mathcal{B}$ computes:

$$R(s)G_1 = \sum_{i=0}^{\deg(R)} r_i(s^iG_1)$$

where $R(x) = \sum_{i=0}^{\deg(R)} r_i x^i$. The reduction $\mathcal{B}$ outputs $(0, R(s)G_1)$ as the solution to the q-SDH problem. This is valid because:

- The polynomial relation $Q(x) = (x - s)R(x)$ with $Q(s) = 0$ and $Q(x) \neq 0$ demonstrates an algebraic dependency on the secret s
- The value $R(s) = \frac{Q(s)}{s-s}$ represents an indeterminate form whose computation requires breaking the q-SDH assumption
- The solution $(0, R(s)G_1)$ explicitly violates the q-SDH hardness property

Success Probability. The reduction succeeds with advantage $\epsilon(\lambda)$ because:

- The SRS distribution is identical to the q-SDH challenge
- Polynomial division is deterministic given $\mathcal{A}$'s output
- Every valid binding collision produces a q-SDH solution

$$\Pr[\mathcal{B} \text{ solves } q\text{-SDH}] = \Pr[\mathcal{A} \text{ breaks binding}] = \epsilon(\lambda)$$

Contradiction. This contradicts the q-SDH assumption since $\epsilon(\lambda)$ is non-negligible. Therefore, no PPT adversary can break the binding property of KZG commitments with non-negligible probability. □

B Proof of Theorem 3

Proof. Suppose there exists a PPT adversary $\mathcal{A}$ that produces a forged proof $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \tilde{\pi})$, accepted by the verifier despite $\mathbf{C} \neq \mathbf{A} \times \mathbf{B}$, with non-negligible probability $\epsilon(\lambda)$. We construct a reduction $\mathcal{B}$ that breaks the q-SDH assumption by leveraging $\mathcal{A}$ as a subroutine.

Reduction Setup: Given a q-SDH challenge tuple $(G_1, sG_1, \ldots, s^qG_1, G_2, sG_2)$, the reduction $\mathcal{B}$ uses this tuple as the structured reference string SRS, and forwards it to $\mathcal{A}$.

Adversarial Output: The adversary returns matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ and forged proof $\tilde{\pi} = (V_{\mathbf{a}_y}, V_{\mathbf{b}_y}, V_\mu, V_W)$, such that the verifier accepts the proof and yet $\mathbf{C} \neq \mathbf{A} \times \mathbf{B}$. Let $y \in \mathbb{F}_p$ denote the evaluation point derived during the protocol execution. Define the following polynomials:

$$P_{\mathbf{a}_y}(x) = \mathrm{Poly}(y_L^\top \mathbf{A}), \quad P_{\mathbf{b}_y}(x) = \mathrm{Poly}(\mathbf{B}y_R), \quad P_\mu(x) = \mathrm{Poly}(y_L^\top \mathbf{C} y_R).$$

Then, define the polynomial:

$$Q(x) := P_\mu(x) - P_{\mathbf{a}_y}(x) \cdot P_{\mathbf{b}_y}(x).$$

Since $\mathbf{C} \neq \mathbf{A} \times \mathbf{B}$, we have $Q(x) \not\equiv 0$, and in particular $Q(y) \neq 0$. The adversary must have computed the witness polynomial:

$$W(x) := \frac{Q(x)}{x - y}.$$

The pairing-based verification condition:

$$e\big(V_W, (s-y)G_2\big) = e\big(V_\mu - V_{\mathbf{a}_y} V_{\mathbf{b}_y}, G_2\big)$$

implies, using bilinearity and the SRS encoding:

$$W(s)G_1 = \frac{Q(s)}{s-y} G_1.$$

Extracting the q-SDH Solution: From the above, the reduction can compute:

$$Q(s) = (s-y)W(s).$$

Since $Q(x)$ and $(x-y)$ are coprime (as $Q(y) \neq 0$), the Extended Euclidean Algorithm yields polynomials $A(x), B(x)$ such that:

$$A(x)Q(x) + B(x)(x-y) = 1.$$

Evaluating at $x = s$:

$$A(s)Q(s) + B(s)(s-y) = 1 \quad \Rightarrow \quad (s-y)(A(s)W(s) + B(s)) = 1.$$

Hence,

$$\frac{1}{s-y} = A(s)W(s) + B(s),$$

and using the SRS:

$$\frac{1}{s-y} G_1 = A(s)W(s)G_1 + B(s)G_1.$$

This yields a valid solution $(y, \frac{1}{s-y}G_1)$ to the q-SDH problem.

Success Probability: The degree of $Q(x)$ is at most $2n-2$. By the Schwartz–Zippel lemma:

$$\Pr[Q(s) = 0] \leq \frac{2n-2}{p},$$

which is negligible. Therefore, if $\mathcal{A}$ succeeds with probability $\epsilon(\lambda)$, then the reduction $\mathcal{B}$ solves the q-SDH problem with probability at least $\epsilon(\lambda) \cdot \left(1 - \frac{2n-2}{p}\right)$, contradicting the q-SDH assumption. □

C Proof of Theorem 6

Proof. **1. Perfect Completeness:** Assume $\mathbf{C}_i = \mathbf{A}_i \times \mathbf{B}_i$ for all $i \in [t]$. The prover:

- Encodes each matrix into polynomials $P_{\mathbf{A}_i}(x), P_{\mathbf{B}_i}(x), P_{\mathbf{C}_i}(x)$.
- Computes KZG commitments $V_{\mathbf{A}_i}, V_{\mathbf{B}_i}, V_{\mathbf{C}_i}$.
- Computes challenge $y_i \leftarrow \mathcal{H}(\text{pp} \parallel V_{\mathbf{A}_i} \parallel V_{\mathbf{B}_i} \parallel V_{\mathbf{C}_i}) \bmod p$, constructs vectors $\mathbf{y}_L^i, \mathbf{y}_R^i$, and forms the witness:
$$W_i(x) = \frac{\mu_i - P_{\mathbf{a}_y^i}(x) P_{\mathbf{b}_y^i}(x)}{x - y_i}$$
where $\mu_i = \mathbf{y}_L^{i\top} \mathbf{C}_i \mathbf{y}_R^i$, and $P_{\mathbf{a}_y^i}(x)$, $P_{\mathbf{b}_y^i}(x)$ interpolate $\mathbf{A}_i^\top \mathbf{y}_L^i$, $\mathbf{B}_i \mathbf{y}_R^i$
- Aggregates using $\rho \xleftarrow{\$} \mathbb{F}_p$:

$$V_{\mathbf{A}}^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} V_{\mathbf{A}_i}, \quad V_{\mathbf{B}}^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} V_{\mathbf{B}_i}$$
$$D^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} \left(V_{\mu_i} - V_{\mathbf{a}_y^i} \cdot V_{\mathbf{b}_y^i} \right)$$
$$V_W^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} V_{W_i}$$

For honest provers, $\mu_i = P_{\mathbf{a}_y^i}(y_i) P_{\mathbf{b}_y^i}(y_i)$, so:

$$V_{\mu_i} - V_{\mathbf{a}_y^i} \cdot V_{\mathbf{b}_y^i} = \mu_i G_1 - P_{\mathbf{a}_y^i}(s) P_{\mathbf{b}_y^i}(s) G_1 = (s - y_i) V_{W_i}$$

Homomorphic aggregation gives:

$$D^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} (s - y_i) V_{W_i} = \left(s - \sum_{i=1}^{t} \rho^{i-1} y_i \right) V_W^{\text{batch}} = (s - z^{\text{batch}}) V_W^{\text{batch}}$$

where $z^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} y_i$. Thus, verification holds:

$$e(V_W^{\text{batch}}, (s - z^{\text{batch}}) G_2) = e(D^{\text{batch}}, G_2)$$

2. Perfect SHVZK: Note on ZK: Achieves Special Honest Verifier Zero-Knowledge (SHVZK). Full ZK in non-interactive setting via Fiat-Shamir requires random oracle programming analysis.

The simulator $\mathcal{S}$ (given public inputs $V_{\mathbf{A}_i}, V_{\mathbf{B}_i}, V_{\mathbf{C}_i}$, challenges ρ, y_i, and SRS $(G_1, sG_1, \ldots, s^q G_1, G_2, sG_2)$):

- Computes $z^{\text{batch}} = \sum_{i=1}^{t} \rho^{i-1} y_i$
- Samples random $r \xleftarrow{\$} \mathbb{F}_p$
- Sets $V_W^{\text{batch}} = rG_1$
- Computes $D^{\text{batch}} = r \cdot \left(sG_1 - z^{\text{batch}} G_1 \right)$
- Outputs $\left(V_{\mathbf{A}}^{\text{batch}}, V_{\mathbf{B}}^{\text{batch}}, D^{\text{batch}}, V_W^{\text{batch}} \right)$

The distribution is identical to real proofs due to:

- V_W^{batch} uniform in $\mathbb{G}_1$ (KZG hiding)

- D^{batch} satisfies verification equation by construction

3. Computational Soundness: If an adversary forges an accepting batch proof where some $\mathbf{C}_j \neq \mathbf{A}_j \times \mathbf{B}_j$:

- By Theorem 3, for the false instance j: $V_{\mu_j} - V_{\mathbf{a}_y^j} \cdot V_{\mathbf{b}_y^j} \neq (s - y_j)V_{W_j}$
- Define the polynomial:

$$F(\rho) = \sum_{i=1}^{t} \rho^{i-1} \left((V_{\mu_i} - V_{\mathbf{a}_y^i} \cdot V_{\mathbf{b}_y^i}) - (s - y_i)V_{W_i} \right)$$

- $F(\rho)$ is non-zero with degree at most $t-1$, so for random ρ: $\Pr[F(\rho) = 0] \leq \frac{t-1}{p}$
- When $F(\rho) \neq 0$, extraction in the Algebraic Group Model yields a q-SDH solution following the same technique as in Theorem 3.

Using this identity and applying the extended Euclidean algorithm to isolate $\frac{1}{s-c}G_1$, one can solve the q-SDH problem as in Theorem 3.

Adversary advantage is bounded by:

$$\epsilon(\lambda) \leq \mathbf{Adv}_{q\text{-SDH}}(\mathcal{B}) + \frac{t-1}{p}$$

The simulator does not require knowledge of the trapdoor s, ensuring that the simulation remains valid under the zero-knowledge property. We conclude that the Batch *zkMaP* protocol offers a compelling trade-off between succinctness, security, and computational cost, making it a promising candidate for practical SNARK-based matrix relations. □

D Parallelization Performance

Table 4: Proving and verification time across different thread counts (in seconds).

Matrix Size	Threads	Proof Time (s)	Verification Time (s)	Speedup (×)
128	1	3.948	0.00368	1.00
	4	1.017	0.00367	3.88
	8	0.532	0.00366	7.42
	16	0.317	0.00383	12.44
	48	0.237	0.00366	16.68
256	1	16.022	0.00368	1.00
	4	4.077	0.00368	3.93
	8	2.073	0.00364	7.73
	16	1.170	0.00368	13.69
	48	0.901	0.00383	17.78
512	1	66.136	0.00373	1.00
	4	16.633	0.00372	3.98
	8	8.367	0.00379	7.90
	16	4.391	0.00385	15.06
	48	3.017	0.00382	21.92
1024	1	287.392	0.00378	1.00
	4	72.065	0.00386	3.99
	8	36.084	0.00383	7.96
	16	18.242	0.00369	15.75
	48	12.502	0.00374	22.99

E Algorithm 2

Algorithm 2 *zkMaP*: Zero-Knowledge Succinct Non-Interactive Proof for Matrix Multiplication

Require: Matrices $\mathbf{A}, \mathbf{B} \in \mathbb{F}_p^{n \times n}$, $\mathbf{C} = \mathbf{A} \times \mathbf{B}$;
SRS $\{sG_1, s^2G_1, \ldots, s^{n^2-1}G_1\}, \{G_2, sG_2\}$, public parameters pp
Ensure: Proof $\pi = (V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, V_{\mathbf{a}_y}, V_{\mathbf{b}_y}, V_\mu, V_W)$
procedure CommitMatrices
  Construct polynomials:

$$P_{\mathbf{A}}(x) \leftarrow \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathbf{A}[i,j] x^{in+j}$$

  Similarly construct $P_{\mathbf{B}}(x)$, $P_{\mathbf{C}}(x)$
  Compute commitments:

$$V_{\mathbf{A}} \leftarrow P_{\mathbf{A}}(s)G_1,\ V_{\mathbf{B}} \leftarrow P_{\mathbf{B}}(s)G_1,\ V_{\mathbf{C}} \leftarrow P_{\mathbf{C}}(s)G_1$$

end procedure
procedure ChallengeGeneration
  Compute $y \leftarrow \mathcal{H}(\text{pp} \parallel V_{\mathbf{A}} \parallel V_{\mathbf{B}} \parallel V_{\mathbf{C}}) \bmod p$
  Define vectors:

$$\mathbf{y}_L \leftarrow (1, y^n, \ldots, y^{(n-1)n})^\top,\ \mathbf{y}_R \leftarrow (1, y, \ldots, y^{n-1})^\top$$

end procedure
procedure ProverComputation
  Compute projections: $\mathbf{a}_y \leftarrow \mathbf{y}_L^\top \mathbf{A},\ \mathbf{b}_y \leftarrow \mathbf{B}\mathbf{y}_R$
  Interpolate $P_{\mathbf{a}_y}(x)$, $P_{\mathbf{b}_y}(x)$ from $\mathbf{a}_y$, $\mathbf{b}_y$
  Commitments:
  $V_{\mathbf{a}_y} \leftarrow P_{\mathbf{a}_y}(s)G_1,\ V_{\mathbf{b}_y} \leftarrow P_{\mathbf{b}_y}(s)G_1$
  Compute $\mu \leftarrow \mathbf{y}_L^\top \mathbf{C} \mathbf{y}_R$
  $V_\mu \leftarrow \mu G_1$
  Construct witness polynomial:

$$W(x) \leftarrow \frac{\mu - P_{\mathbf{a}_y}(x) \cdot P_{\mathbf{b}_y}(x)}{x - y}$$

  $V_W \leftarrow W(s)G_1$
  return $\pi \leftarrow (V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, V_{\mathbf{a}_y}, V_{\mathbf{b}_y}, V_\mu, V_W)$
end procedure
procedure Verify(pp, π)
  Parse $\pi = (V_{\mathbf{A}}, V_{\mathbf{B}}, V_{\mathbf{C}}, V_{\mathbf{a}_y}, V_{\mathbf{b}_y}, V_\mu, V_W)$
  Compute $y \leftarrow \mathcal{H}(\text{pp} \parallel V_{\mathbf{A}} \parallel V_{\mathbf{B}} \parallel V_{\mathbf{C}}) \bmod p$
  Verify pairing equation:

$$e(V_W, sG_2 - yG_2) \stackrel{?}{=} e(V_\mu, G_2) \cdot e(V_{\mathbf{a}_y}, G_2)^{-1} \cdot e(V_{\mathbf{b}_y}, G_2)^{-1}$$

  if equality holds **then return** Accept
  elsereturn Reject
  end if
end procedure

F Algorithm 3

Algorithm 3 Batch Zero-Knowledge Matrix Multiplication Proof (zkMaP)

Require: Matrix pairs $\{(\mathbf{A}_i, \mathbf{B}_i)\}_{i=1}^{t}$, where $\mathbf{A}_i, \mathbf{B}_i \in \mathbb{F}_p^{n \times n}$;
Products $\mathbf{C}_i = \mathbf{A}_i \times \mathbf{B}_i$ for $i \in [1, t]$;
KZG SRS $(G_1, sG_1, \ldots, s^q G_1, G_2, sG_2)$ with trapdoor $s \in \mathbb{F}_p$;
Hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{F}_p$
Ensure: Zero-knowledge proof for $\{\mathbf{C}_i = \mathbf{A}_i \times \mathbf{B}_i\}_{i=1}^{t}$
procedure SETUP
  for $i \in [1, t]$ **do**
    Flatten $\mathbf{A}_i, \mathbf{B}_i, \mathbf{C}_i$ row-wise to vectors in $\mathbb{F}_p^{n^2}$
    Map to polynomials: $P_{\mathbf{A}_i}(x), P_{\mathbf{B}_i}(x), P_{\mathbf{C}_i}(x) \in \mathbb{F}_p[x]$
    Commit: $V_{\mathbf{A}_i} \leftarrow P_{\mathbf{A}_i}(s)G_1, \quad V_{\mathbf{B}_i} \leftarrow P_{\mathbf{B}_i}(s)G_1, \quad V_{\mathbf{C}_i} \leftarrow P_{\mathbf{C}_i}(s)G_1$
  end for
end procedure
procedure GENERATEINDIVIDUALPROOFS
  for $i \in [1, t]$ **do**
    Challenge: $y_i \leftarrow \mathcal{H}(\text{pp} \parallel V_{\mathbf{A}_i} \parallel V_{\mathbf{B}_i} \parallel V_{\mathbf{C}_i}) \bmod p$
    Construct vectors: $\mathbf{y}_L^i \leftarrow (1, y_i^n, \ldots, y_i^{(n-1)n})^\top$, $\mathbf{y}_R^i \leftarrow (1, y_i, \ldots, y_i^{n-1})^\top$
    Compute projections: $\mathbf{a}_y^i \leftarrow (\mathbf{y}_L^i)^\top \mathbf{A}_i$, $\mathbf{b}_y^i \leftarrow \mathbf{B}_i \mathbf{y}_R^i$
    Interpolate: $P_{\mathbf{a}_y^i}(x), P_{\mathbf{b}_y^i}(x)$ from $\mathbf{a}_y^i, \mathbf{b}_y^i$
    Commit: $V_{\mathbf{a}_y^i} \leftarrow P_{\mathbf{a}_y^i}(s)G_1$, $V_{\mathbf{b}_y^i} \leftarrow P_{\mathbf{b}_y^i}(s)G_1$
    Compute: $\mu_i \leftarrow (\mathbf{y}_L^i)^\top \mathbf{C}_i \mathbf{y}_R^i$, $V_{\mu_i} \leftarrow \mu_i G_1$
    Witness:

$$W_i(x) \leftarrow \frac{\mu_i - P_{\mathbf{a}_y^i}(x) \cdot P_{\mathbf{b}_y^i}(x)}{x - y_i}$$

    Commit: $V_{W_i} \leftarrow W_i(s)G_1$
  end for
end procedure
procedure AGGREGATEBATCHPROOF
  $\rho \xleftarrow{\$} \mathbb{F}_p$
  $V_{\mathbf{A}}^{\text{batch}} \leftarrow \sum_{i=1}^{t} \rho^{i-1} V_{\mathbf{A}_i}, \quad V_{\mathbf{B}}^{\text{batch}} \leftarrow \sum_{i=1}^{t} \rho^{i-1} V_{\mathbf{B}_i}$
  $D^{\text{batch}} \leftarrow \sum_{i=1}^{t} \rho^{i-1} (V_{\mu_i} - V_{\mathbf{a}_y^i} \cdot V_{\mathbf{b}_y^i})$
  $V_W^{\text{batch}} \leftarrow \sum_{i=1}^{t} \rho^{i-1} V_{W_i}$
  return $(V_{\mathbf{A}}^{\text{batch}}, V_{\mathbf{B}}^{\text{batch}}, D^{\text{batch}}, V_W^{\text{batch}})$
end procedure
procedure VERIFY(BatchProof)
  Compute: $z^{\text{batch}} \leftarrow \sum_{i=1}^{t} \rho^{i-1} y_i$
  Verify pairing:

$$e(V_W^{\text{batch}}, (s - z^{\text{batch}})G_2) \stackrel{?}{=} e(D^{\text{batch}}, G_2)$$

  if holds **then**
    return Accept
  else
    return Reject
  end if
end procedure